

SmartLearner

Adaptive Multi-Stage Multi-Agent Learning System

Final Project Report Submitted to

The Department of Computer Science

Faculty of Computer and Information Technology

Jordan University of Science and Technology

In Partial Fulfillment of the Requirements for the Degree of

Bachelors of Science in

Artificial intelligence and Computer Science

By

Manar Mahmoud Al Nashash (164070)

Omar Adel AlTamimi (154753)

Mohammed Hatem Nayef Atwah (156179)

Supervised by : Dr. Rasha Obeidat

تعود حقوق الملكية الفكرية لمشاريع التخرج ونتائجها (مثل براءات الاختراع أو أي منتج قابل للتسويق) إلى جامعة العلوم والتكنولوجيا الأردنية، وتخضع هذه الحقوق إلى قوانين وأنظمة و تعليمات الجامعة المتعلقة بالملكية الفكرية وبراءات الاختراع.
بناءا على ما سبق أوافق على ما يلي:

1) أن أحفظ كافة حقوق الملكية الفكرية لجامعة العلوم والتكنولوجيا الأردنية في مشروع التخرج.
2) أن ألتزم بوضع اسم جامعة العلوم والتكنولوجيا الأردنية و أسماء جميع الباحثين المشاركين في المشروع على أي نشرة علمية للمشروع كاملا أو لنتائجه. و يشمل ذلك النشر في المجلات و المؤتمرات العلمية عامة أو النشر على المواقع الإلكترونية أو براءات الاختراع أو المسابقات العلمية.
3) أن ألتزم بأسس حقوق التأليف المعتمدة في جامعة العلوم والتكنولوجيا الأردنية.
4) أن أقوم بإعلام الجهة المختصة في الجامعة عن أي اختراع أو اكتشاف قد ينتج عن هذا المشروع و أن ألتزم السرية التامة في ذلك و أن أعمل من خلال الجامعة على الحصول على براءة الاختراع التي قد تنتج عن هذا المشروع.
5) أن تكون جامعة العلوم والتكنولوجيا الأردنية هي المالك لأي براءة اختراع قد تنتج عن هذا المشروع و تشمل هذه الملكية حق الجامعة في إعطاء التراخيص و التسويق و البيع كمؤسسة راعية و داعمة لكافة الأنشطة البحثية. و يكون حق للطالب شمول اسمه على براءة الاختراع كأحد المخترعين، و في حال تم إعطاء تراخيص أو تسويق و بيع لأي من منتجات المشروع يمنح المخترعون بما فيهم الطالب نسبة من الإيرادات حسب تعليمات البحث العلمي في جامعة العلوم والتكنولوجيا الأردنية.

اسم الطالب : منار محمود النشاش **التوقيع :**

اسم الطالب : عمر عادل التميمي **التوقيع :**

اسم الطالب : محمد حاتم عطوة **التوقيع :**

اسم المشرف : رشا عبيدات التوقيع

ABSTRACT

We, as students, always experienced the frustration of studying difficult subjects. In this case, the problem does not lie in the insufficient amount of educational materials available, but in the difficulties of finding suitable material that corresponds to the learner's level, especially in the case of complex topics and constant need for guidance. Educational resources available today contain static non-adaptive information which often results in inefficient learning due to excessive confusion in understanding concepts that were not previously introduced at the necessary level.

In order to address this issue, we have come up with "SmartLearner", a novel learning platform based on the Mullti Staging Multi-Agent System (MAS). Unlike existing approaches, where a single general artificial intelligence tries to perform all the tasks related to the education process, in SmartLearner agents specialize in performing certain actions. Some examples include assessing the learner's level through a test, generation of an ideal personalized syllabus, generation of dynamic lessons using RAG approach, evaluation of explanations with help of validator, and evaluation of performance.

Thus, we have implemented a specific workflow, where generator agents generate educational materials, while validator agents assess their quality. In such a way, the educational material presented to the learner has been guaranteed to be completely correct in terms of top chunks and resources embeddings. All this workflow is organized in the SmartLearner through centralized management carried out with help of the MCP server. The MCP server organizes communication between agents, updating the learner's mastery state through interaction logs kept in our relational database in order to switch topics.

We expect to see the following results: improved learner's comprehension, decreased learning time, and increased effectiveness of the entire process.

ABSTRACT 3

CHAPTER 1 ..... **Error! Bookmark not defined.**
INTRODUCTION ..... **Error! Bookmark not defined.**
1.1 Motivation ..... **Error! Bookmark not defined.**
1.2 Problem Statement ..... **Error! Bookmark not defined.**
1.3 Functionalities of the Project ..... **Error! Bookmark not defined.**
1.4. How Do Agents Communicate? ..... **Error! Bookmark not defined.**
1.5 Summary of the Results ..... **Error! Bookmark not defined.**
Chapter 2 ..... **Error! Bookmark not defined.**
REVIEW OF RELATED LITERATURE ..... **Error! Bookmark not defined.**
2.1 Comparison with Existing Platforms ..... **Error! Bookmark not defined.**
2.2 Related Research ..... **Error! Bookmark not defined.**
Chapter 3 16
ANALYSIS AND DESIGN 16
3.1 Project Analysis 16
3.1.1How we can improve the System and Stats 19
3.2 Functional Requirements 19
3.3 Non-Functional Requirements 20
3.4 System Architecture Design 21
3.5 UML Diagrams 23
CHAPTER 4 ..... **Error! Bookmark not defined.**
IMPLEMENTATION ..... **Error! Bookmark not defined.**
4.1 Agent Logic Implementation ..... **Error! Bookmark not defined.**
4.2 The MCP Orchestrator ..... **Error! Bookmark not defined.**
4.3 Context Window ..... **Error! Bookmark not defined.**
4.4 Structural Implementation and Data Design ..... **Error! Bookmark not defined.**
4.5 Challenges Encountered and Proposed Solutions ..... **Error! Bookmark not defined.**
Chapter 5 ..... **Error! Bookmark not defined.**
5.1 Evaluation Approaches for AI Agents ..... **Error! Bookmark not defined.**
5.2 Human Evaluation as a Core Evaluation Method ..... **Error! Bookmark not defined.**
5.3 Evaluation Scope and Limitations ..... **Error! Bookmark not defined.**
5.4 Summary of the Evaluation Perspective ..... **Error! Bookmark not defined.**
Chapter 6 ..... **Error! Bookmark not defined.**
Conclusion ..... **Error! Bookmark not defined.**
6.1 Key Contributions and Achievements ..... **Error! Bookmark not defined.**
6.2 Validation of the Proposed Direction ..... **Error! Bookmark not defined.**
6.3 Scope of Project 1 and Future Development **Error! Bookmark not defined.**

CHAPTER 1

INTRODUCTION

Education of such modern developing technologies as Artificial Intelligence (AI), Natural Language Processing (NLP), Computer Vision (CV), and Deep Learning can become difficult due to various reasons. In particular, people often face the problem of finding proper educational materials adjusted to their level of competency. It is known that students find it quite difficult to cope with thick books, and even more, learn various information from various unstructured sources online. Besides, traditional methods of education cannot provide much flexibility and personalization in regard to each individual participant of the educational process. In addition, there is no way to use some advanced online platforms as they lack personalization and interactivity.

In order to address the shortcomings of current educational methods, a brand-new system named SmartLearner has been designed. Instead of adopting a certain Artificial Intelligence solution as a tool for managing the whole process, our system operates on Multi-Agent Orchestration Architecture. In other words, our platform includes some unique agents responsible for performing different actions related to education. Thus, they are able to create new educational materials, test students' performance, retrieve grounded educational material through the RAG approach, and adjust the educational process on that basis.

The heart of SmartLearner implementation is the centralized Orchestrator Service providing for the communication between all other components through the hub-and-spoke model. Execution logs generated by every component are accumulated to create the audit trail. Thus, our Analytics Agent can constantly monitor the learners' progress.

Currently, SmartLearner supports two learning tracks (Python programming and Deep Learning). However, the design of our system allows for extending its architecture by covering new domains such as NLP and CV.

All these system components – adaptive testing, syllabus creation, RAG-based lessons generation, deterministic analysis, and real-time adaptation – are essential in creating the conditions where each user receives the individually adapted course.

1.1 Motivation

The motivation for this work comes from deficiencies of currently available learning tools and approaches in technical disciplines. Students find it hard to get the necessary materials with a gradual increase in knowledge corresponding to one's level of understanding. In result, many have to study complex scientific textbooks and extensive web material, which can be cognitively difficult to process, slowing down learning process.

Current education solutions are usually expensive and rigid, providing little room for adjustment and personalization. In contrast, most online learning platforms offer static materials without adequate adaptivity features and personalization algorithms. Moreover, most existing educational

platforms lack a possibility to personalize a learning process according to one's strengths, difficulties, rhythm, and progress.

Based on our experience as students studying technical topics, we managed to define the areas where improvement is necessary through iterations and implementation of learning platform. We found the necessity of building an intelligent platform capable of adaptive behavior based on learner's characteristics and constant evaluation of his performance.

SmartLearner solves this problem by introducing Multi-Agent educational architecture where special agents carry out tasks such as placement testing, syllabus formation, lessons generation, questioning, validation, exams, and analytics.

1.2 Problem Statement

The main issue that will be considered in the course of this project is the lack of intelligent and adaptive educational systems able to recognize the learner, generate the appropriate learning materials, and adjust the learning path based on the learner’s progress and mastery.

Many existing educational solutions tend to offer content that may be overly complex, overly general, or unrelated to the actual level of the learner's knowledge. These problems lead to inefficiency, low learner engagement, frustration, and poor educational results. Besides, many solutions do not offer the possibility of carrying out sophisticated behavioral analysis of the learner, offering context-based explanations and adjusting the assessment and recommendation process in real-time.

Moreover, depending on one AI model to perform all educational activities might affect reliability and specialization of the process, as various activities like content creation, validation, analytics, and question answering require specific reasoning and validation procedures.

Finally, current solutions are seldom based on the integration of determinative learning analytics methods such as Deep Knowledge Tracing (DKT) and LLM-generated content. As a consequence, analytics becomes unreliable and explanations produced by LLMs become disconnected and ungrounded.

Thus, the proposed project will involve a coordinated Multi-Agent system with specialized agents working within a centralized orchestration infrastructure.

1.3 Functionalities of the Project

The deployed SmartLearner platform offers the following functionalities with the help of Multi-Agent approach coordination:

- **Placement Agent (Generator + Validator):** Performs the adaptive multi-level placement test with four difficulty levels (Beginner, Intermediate, Advanced, Very Advanced). The agent evaluates learners' performance and builds the initial profile.
- **Syllabus Agent (Generator + Validator):** Creates an individualized learning plan according to the learner's performance in the placement test and selected learning path (Python for Everybody (PY4E) or Deep Learning).
- **Lesson Agent (Generator + Validator):** Generates the content of the lessons which includes the description, examples of the code, exercises and quizzes. The lessons are grounded on the RAG-retrieved educational material.
- **RAG Service:** Retrieves semantically relevant content chunks of the embedded educational sources with the use of vector similarity search for grounding and explaining.
- **Q&A Agent (Generator + Validator):** Answers the learners' questions by retrieving contextually meaningful information and generating responses via LLM. The response is validated before being provided to the user.
- **Exam Agent (Generator + Validator):** Automatically generates adaptive assessments for the learners and evaluates their performance.
- **Analytics Agent:** Tracks the learner's progress with the help of Deep Knowledge Tracing (DKT) method, computes the level of mastery of the learner, detects weak spots and risk factors, calculates learning velocity and makes recommendations regarding the next step.
- **Orchestrator Service:** Represents the centralization point where requests are routed between agents, workflow execution is managed, logs are gathered for every agent run and analytics are integrated into the learning process.

1.4 Agent Communication Architecture

SmartLearner utilizes a hub-and-spoke-based communication architecture that is controlled by the Orchestrator Service. The agents have no direct communication with each other. All communication with agents is carried out via the Orchestrator Service.

After receiving a learner request, the Orchestrator Service detects the type of the request and passes it on to the specialized agent. All agent executions are logged in the database audit trail for analytics and history purposes.

Consider the following communication example within the course of a question-answer exchange.

1. A learner asks a question, e.g., “Explain CNNs.”

2. The request is passed to the backend API endpoint.

3. The Orchestrator Service receives a request along with an authentication context of the learner.

4. The Orchestrator calls the Q&A Generator Agent and supplies:

othe learner’s query,

othe contextual data obtained with the help of the RAG Service,

ooften, the data about the learner's level of mastery provided by the Analytics Agent.

5. Q&A Generator Agent returns a structured answer with a set of educational content.

6. Orchestrator Service calls the Q&A Validator Agent and passes the generated answer for verification.

7. Both agent executions are logged into the database.

8. The verified answer is sent back to the front-end and delivered to the learner.

This architecture brings more flexibility, reliability, and easier maintenance.

1.5 Results Summary

The designed SmartLearner solution yielded the following results:

1.Personalized Learning Trajectories:

The solution creates adaptive curricula and lesson planning suited to varying learner proficiency levels, from novice to highly advanced learners.

2.Cognitive Overhead Reduction:

Explanations based on RAG and a gradually advancing lesson structure make understanding complicated material easier for learners.

3.Performance Tracking:

The Analytics Agent offers tracking of learner mastery, weak spots identification, measuring learning speed, and assessment of learner risks.

4.On-The-Fly Learning Adaptation:

Lesson plans are adjusted according to learner performance in quizzes and exams, and learner interactions with the system.

5.Enhanced Learner Engagement:

Feedback, adaptive learning trajectory, and explanations personalize the learning experience, increasing interactivity and motivation.

6.Full-Stack Integration:

The developed platform incorporates a FastAPI backend, React front end, Supabase PostgreSQL database, RAG pipeline, and Multi-Agent coordination into one educational platform.

Chapter 2

REVIEW OF RELATED LITERATURE

This chapter is about the related works on intelligent learning systems and adaptive learning platforms. Covering the background information and knowledge in the development of the project. Its related work on the learning applications and research studies on personalization, adaptability, and AI assistance in technical learning.

The first part focuses on popular educational applications available in the current market and compare these platforms based on their learning delivery system, learner assessment, and learner engagment methods. We compared them to our application to identify both their advantages as well as limitations that block their efficiency for technological learning.

The second part compares the relevant studies that have been conducted in adaptive learning systems, Multi-Agent systems, Retrieval Augmented Generation, and also automated assesments. We reviewed to determine the areas that have not been addressed by present approaches.

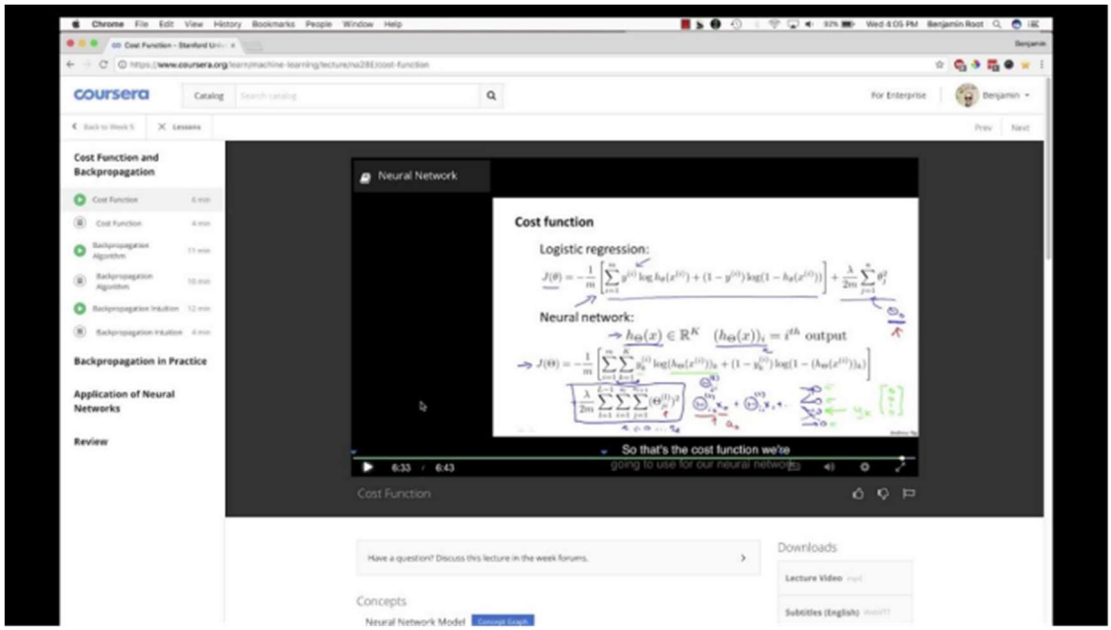
Through an evaluation of existing applications as well as related literature reviews we read, we came up with an ideal system design that explains reasons why some architectural and functional decisions were made in our project.

2.1 Comparison with Existing Platforms

To understand why SmartLearner is truly effiecnt, we compared to other learning platforms and explain where they fall short in technical education:

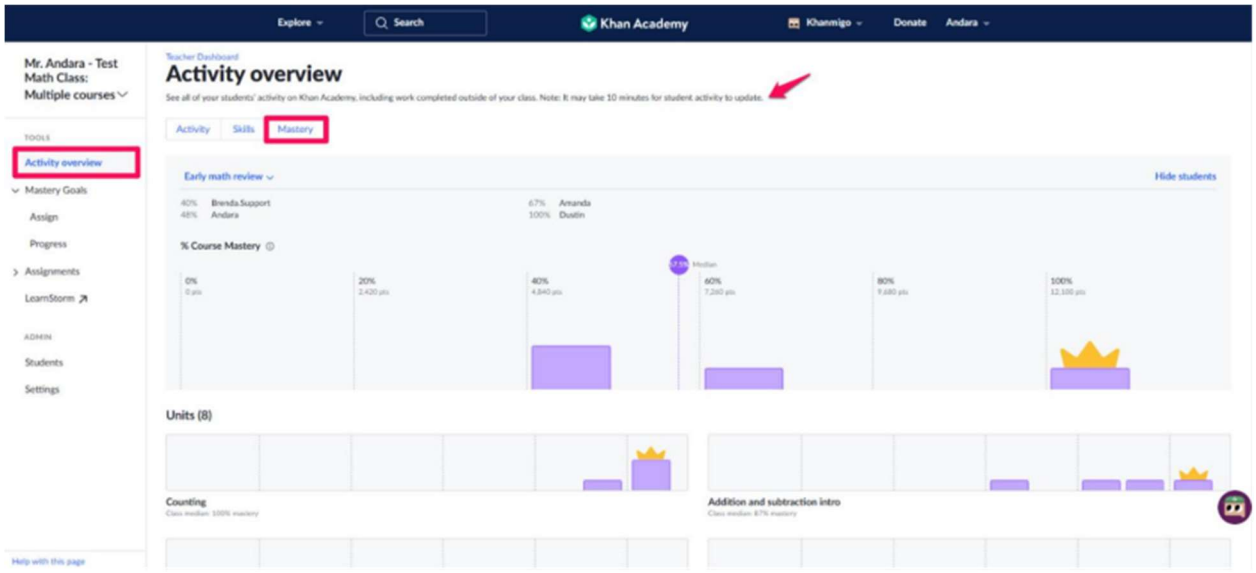
2.1.1 Udemy & Coursera: great libraries of knowledge, but they are "static resources". They provide high quality videos, but they don't know who you are. If you don't understand a concept in minute 5, the video just keeps playing. SmartLearner replaces this passive watching with an active, agentic dialogue that pauses and adapts based on your specific confusion.

2.1.1 Coursera video player interface



2.1.2 Khan Academy: uses "Mastery Learning" to make sure you understand basic concepts before moving on, its limited to K-12 subjects and lacks technical depth for advanced IT tracks. Furthermore, its feedback is rule based. Our project uses LLM driven agents to provide clear, creative explanations and analogies that a rule based system cannot generate.

2.1.2Khan Academy mastery dashboard



2.1.3Duolingo: they don't generate content; they just serve pre-written sentences. SmartLearner is a Generator platform. It doesn't just pick a question from a list; it builds a new one tailored to your specific mastery level and then uses a Validator Agent to ensure that the code it just wrote for you actually works.



2.1.3Duolingo lesson UI word bank

2.2 Related Research

Here we discussed some research articles relevant to the area of intelligent learning systems and artificial intelligence application in general. Mainly, our interest concerned large language models and multi-agent architectures, which we use to identify new opportunities for achieving autonomous reasoning and collaboration of the agents. Simultaneously, we assessed the limitations of applying current methods to developing an educational system. We gained an understanding of possible approaches to our SmartLearner project and obtained certain ideas to implement the basic elements.

2.2.1 Multi-Agent Cooperation and Role-Based Interaction

Recently, there have been more efforts directed at the cooperation between multiple AI agents. In particular, one paper published by Li et al. proposed the development of a role-based model of interactions between chat agents. The idea consists in assigning specific roles and organizing the

interaction procedure based on them. This is expected to reduce conflicts associated with ambiguity or redundancy of agents' actions, as well as endless loops of communication.

It looks like the proposed framework works well. According to the study results, multi-agent systems employing role assignments performed better than those relying solely on agents' prompts in case of complex tasks. What makes the findings especially valuable, in my opinion, is the efficiency that the introduction of a role-based framework seems to generate during collaborative operations.

While focusing mainly on the general issues of task-solving and agent cooperation, the authors failed to discuss any educational aspects, including assessment of learners' skills and quality check of instructional content. These features play an essential role in the proposed solution.

2.2.2 Generative Agents and Human Behavior Simulation

We also looked at a different study that was all about generative agents and how they can imitate what real people do. Park and others (2).They brought in agents that could remember things, think about what they did, and plan ahead to act the same way over a period of time.These agents can keep track of what happened before, remember the stuff that matters, think about how things went, and figure out what to do next.We ran some experiments, and what we saw were pretty lifelike social behaviors, and the agents actually kept their personalities steady.

This work gives us some good ideas about memory and long-term reasoning, but it's really more about social simulations than stuff for schools.The way this is set up, it doesn't really help with making school plans, checking if students are learning automatically, or keeping an eye on how they're doing all the time. These are important things for adaptive learning tools, you know, the ones that are smart like SmartLearner

2.2.3 LLM-Based Agent Architectures

Numerous studies investigated the general concept of utilizing large language models as the main components for agents.Xi et al. Thus, this paper analyzed the development of AI agents throughout history and described their main properties in terms of percepts, cognition, memory, and action. The survey reveals the high effectiveness of using large language models as the kernel component of both single and multi-agent systems.

In general, this paper significantly contributes to a better understanding of the theoretical foundations of LLM-based agents; however, it does not focus specifically on architectures that apply to adaptive learning platforms with a pre-defined learning trajectory. We believe that the lack of such information represents another research gap that should be considered when developing a platform.

2.2.4 Multi-Agent Conversation Frameworks

Wu et al (4) In addition, we explored AutoGen, which is an open-source framework that allows creating multiple agents through conducting conversations. This tool enables developers to create different types of agents, allocate responsibilities among them and control the interaction between these agents through regular programming languages. Based on our empirical research, we discovered that it contributes significantly to improving our logical skills, debugging process, and final result regardless of whether the task involves programming or requires answers to some questions.

In spite of this tool's positive impact on our work, it was not designed for learning purposes. Therefore, it does not contain certain features such as the ability to know a person's level of expertise and his/her progress regarding a specific topic, and assess whether the created content is relevant.

2.2.5 Learning Through Sequential Interaction and Feedback

Apart from agent-based approaches, sequential learning also focuses on the significance of feedback and past interactions. For example, the study by Seshadri et al(5) examines transformers for sequential recommendation tasks and proves that taking both positive and negative feedback into consideration increases adaptability and efficiency.

This approach is not directly associated with the field of education, but still contains essential concepts such as continuous interaction logging, feedback-based adaptation, and sequential learning that are very useful when developing intelligent tutoring systems and mastery-based learning techniques used in SmartLearner.

2.2.6 Research Gap

Despite there being a number of educational tools available online, they mostly rely on either static or only somewhat interactive content. Some may include AI features, but only within one single model and thus cannot provide necessary flexibility.

There seems to be little research conducted on the use of multiple agents in learning systems where new skills are to be taught sequentially and the accuracy and efficiency of generated solutions are to be validated. The need for proper learning sequences, content quality control, students' progress tracking as well as generator-validator workflows, centralized agents coordination, adaptability, and personalized content creation are concepts that need further research. Therefore, after identifying the gap, we developed SmartLearner, integrating our best practices from multi-agent systems based on LLMs.

Our setup brings together special tools for different jobs, like checking things out, creating stuff, making sure it's right, explaining how it works, and seeing how well it performs. Everything is kept in line by a MCP, which makes sure it all works together smoothly. So, SmartLearner sort of takes what's already out there with agent-based task-solving, but makes it real for schools and learning.

SmartLearner	Duolingo	Khan Academy	Udemy / Coursera	**Feature/Capability**
✓	✓	✓	✗	Adaptive lerning path based on learning performance
✓	✗	✗	✗	Personalized syllabus per learner
✓	✗	✗	✗	Dynamic contant generation
✓	✗	✗	✗	Retrieval-Augmented Generation (RAG)
✓	✓	✓	✗	Continuous learner assessment
✓	✓	✓	✗	Mastery tracking over time
✓	✗	✗	✗	Immediate, context-aware feedback
✓	✗	✗	✗	Multi-Agent architecture
✓	✗	✗	✗	Generator–Validator mechanism
✓	✓	✓	✗	Curriculum-aware responses

Table 2.2.6 Comparison of SmartLearner with Existing Learning Platforms

Chapter 3

ANALYSIS AND DESIGN

The AI-based learning system analysis and design using UML diagrams such as use-case diagram, class diagram, sequence diagram, and ER diagram are presented in this chapter.

The design of our system comprises the usage of various AI-based techniques (**Multi-Agent System, MAS**) controlled by the main node named Model Context Protocol (**MCP**) Server. It describes the structure of the system, data flow and agent behavior within the system. The idea of this design is to create a stable, quick and intelligent learning system.

3.1 Project Analysis

Data from a survey regarding online vs. in-person training preferences. By analyzing, we can derive actionable insights to improve the SmartLearner system and its accuracy.

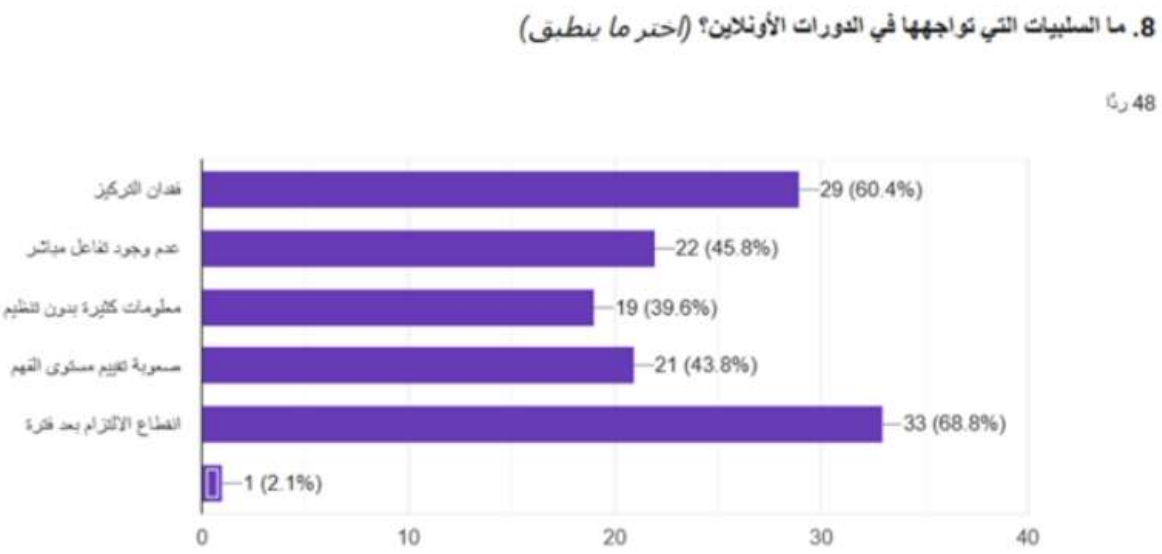


Figure 3.1 Survey form used in this study

The top three challenges of online courses are:

1. **Interruption of commitment after a period** (33 respondents).
2. **Loss of focus** (29 respondents).
3. **Difficulty assessing the level of understanding** (21 respondents).

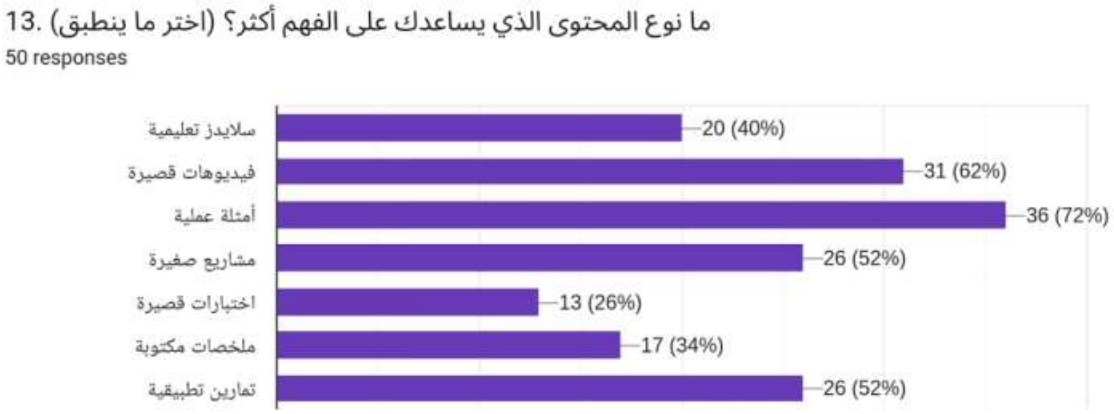
The most important advantages of online courses are:

1. **"Freedom of time"** (35 respondents).
2. **"Ability to repeat lessons more than once"** (35 respondents)
3. **Progress tracking** (31 respondents)

The most important advantages of in-person courses are:

1. **"Ease of interaction with the instructor"** (35 respondents).
2. **"Academic atmosphere encourages focus"** (27 respondents).
3. **"Possibility of asking direct questions"** (26 respondents).

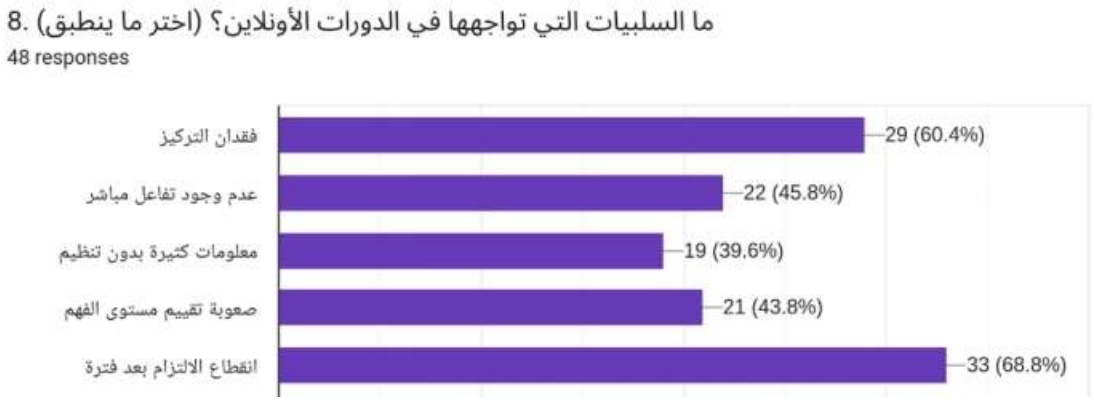
Technical Analysis & Quantifiable Improvement



1.Improve Interaction and Flexibility (Syllabus Agent)

- Average satisfaction with face-to-face education (3.72/5) is greater than satisfaction with online education (3.47/5). In case of face-to-face education, the most significant positive aspects are interaction; "Ease of interaction with the trainer"(34) is the highest rated one. In the case of online education, the most significant positive aspects are flexibility, such as "time freedom" (34) and "opportunity to view the lesson more than once" (34).
- Project application: in order to meet the preference of our customers we use the face-to-face approach by implementing interaction points (like Explanation agent); we emphasize the

freedom of time and an opportunity to revise the lessons.



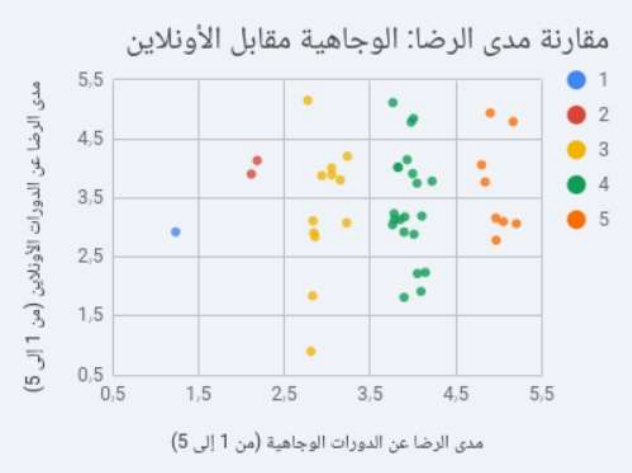
2. Predict Incompletion and Agent Intervention (Analytics Agents)

- Key obstacles related to online learning include commitment and focus issues such as “Disrupted commitment after a period” (mentioned 28 times) and “Loss of focus” (mentioned 24 times).
- Application to Project: If the system identifies that a user is “at risk of not completing,” the Syllabus Agent should take actions including recommendations for a pause, some motivational activity, and/or short videos (the latter mentioned favorably by 27 participants)..

3. Improve Course Quality and Structure Based on User Preferences

•Insight: Primary obstacles to further learning are “unsuitable explanation” (mentioned 28 times) and “lessons accumulation” (mentioned 20 times). To address these barriers, participants favored “content rich in practical examples” (30 mentions) and structural design. Particularly, 31 participants recommended the course to be “divided into levels (beginner – intermediate – advanced)”.

•Application to Project: It is important for the platform to focus on content creation rich in practical examples and divided into levels to avoid lessons accumulation and ensure a rational information flow.



3.1.1How we can improve the System and Stats

1. **Refined Risk Prediction:**
    1. **Implementation:** Get specific features from the survey in the DKT model, specifically the time elapsed since the last login, focus (lesson completion ratio) and retry of quizzes.
    2. **Benefit:** Enhancing the recall of the analytics model. Detecting students that are about to drop because of lost commitment.
2. **Personalized Content:**
    1. **Implementation:**uqse content preferences retrieved through the survey as weights for the RAG model retrieval. If the student's preference is Practical Examples, then increase the retrieval score for all documents that contain studies.
    2. **Benefit:**Increasing student satisfaction and engagement.
3. **Dynamic Difficulty Adjustment:**
    1. **Implementation:** Take the advantage of the survey insight about Unclear Content and provide an efficient feedback loop. If the explanation got low score or the student failed the quiz, then automatically decrease the complexity or change the explanation approach to a more understandable one (such as visualization).
    2. **Benefit:** Decreasing the error rate and improving mastery state stats

3.2 Functional Requirements

3.2.1 Student (User) Functional Requirements

- Students shall be able to register and log into the system.

- Students shall take a placement test to determine their current skill level.
- Students shall be assigned to the proper learning track according to their placement results.
- Students shall have a dynamically generated syllabus and lesson.
- Students shall have access to the learning resources (slides, PDFs, articles).
- Students shall be able to ask in-lesson questions and get AI-generated explanations to their queries.
- Students shall take quizzes and tests generated by the system.
- Students shall see their progress, level of mastery, performance analytics.
- Students shall be suggested a personal study path according to the above data.

3.2.2 Admin Functional Requirements

- Admins shall manage learning tracks and course metadata.
- Admins shall monitor student progress and system analytics.
- Admins shall manage learning resources used for RAG.
- Admins shall view agent interaction logs for auditing and debugging.
- Admins shall control agent configurations and system parameters.

3.2.3 System / AI Functional Requirements

- The system shall orchestrate multiple AI agents through a central MCP server.
- The system shall generate placement tests, syllabus, lessons, and exams.
- The system shall validate generated content using validator agents.
- The system shall retrieve information from a vector database (RAG).
- The system shall store and update student mastery and progress states.
- The system shall log all agent actions and interactions.
- The system shall update analytics after every assessment or interaction
- The system shall use a central MCP (orchestrator) server to route requests to the appropriate agent.

3.3 Non-Functional Requirements

3.3.1 Performance

- The system should respond to student queries within acceptable time
- Content generation and validation should complete within a reasonable explanation
- The system should rate limiting for API requests to prevent system abuse and manage costs depending on tokens

3.3.2 Usability

- The system should be easy to use for students with minimal training
- Navigation between lessons, assessments, and progress tracking should be clear and intuitive.
- System feedback (loading, errors, responses) should be understandable and user-friendly.
- The interface should support a smooth learning experience without unnecessary complexity.

3.3.3 Reliability

- The system should ensure high availability for core learning features.
- Failures in one agent should not crash the whole system.
- The system should support retry and fallback mechanisms for agents.
- Configuration changes (models, prompts, roles) should not require big changes.

3.3.4 Security

- User authentication data must be securely stored and encrypted.
- Role-based access control should distinguish students and admins.
- Only authorized agents may read/write sensitive data
- API Key Management preventing external calls to agents
- IP addresses rate limiting to prevent unauthorized external requests

3.4 System Architecture Design

3.4.1 System Architecture

A central Model Context Protocol (MCP) Server orchestrates and coordinates multiple specialized AI agents. The MCP Server routes user requests based on intent, coordinates content generation, validation, assessment, and analytics agents, and manages interactions with both the relational database and the vector database to ensure adaptive, consistent, and personalized learning workflows.

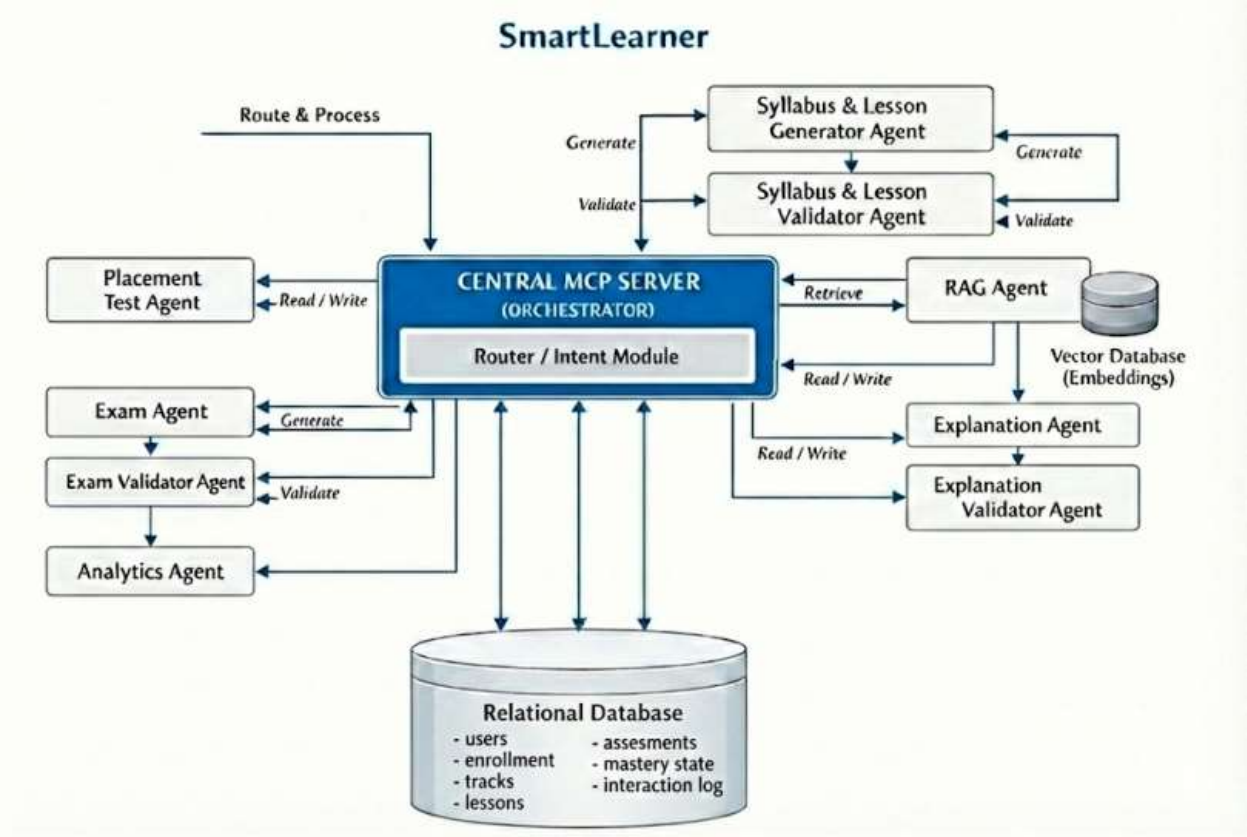


Figure 3.4.1: SmartLearner System Architecture with MCP-Orchestrated Multi-Agent Design

3.4.2Agentic Workflow

This diagram illustrates the agentic workflow of the proposed model. It involves a student's entry point, followed by the transfer of their entry via an intention from the MCP Server. On this basis, the system dispatches the user request to the corresponding specific agent, including the Placement Agent, Content Generation Agent, RAG Agent, or Exam Agent. The retrieved information stored in the vector database will be processed through validation and used for explanation and assessment generation. The interaction results are tracked by the Analytics Agent

to update the mastery state of the learner constantly.

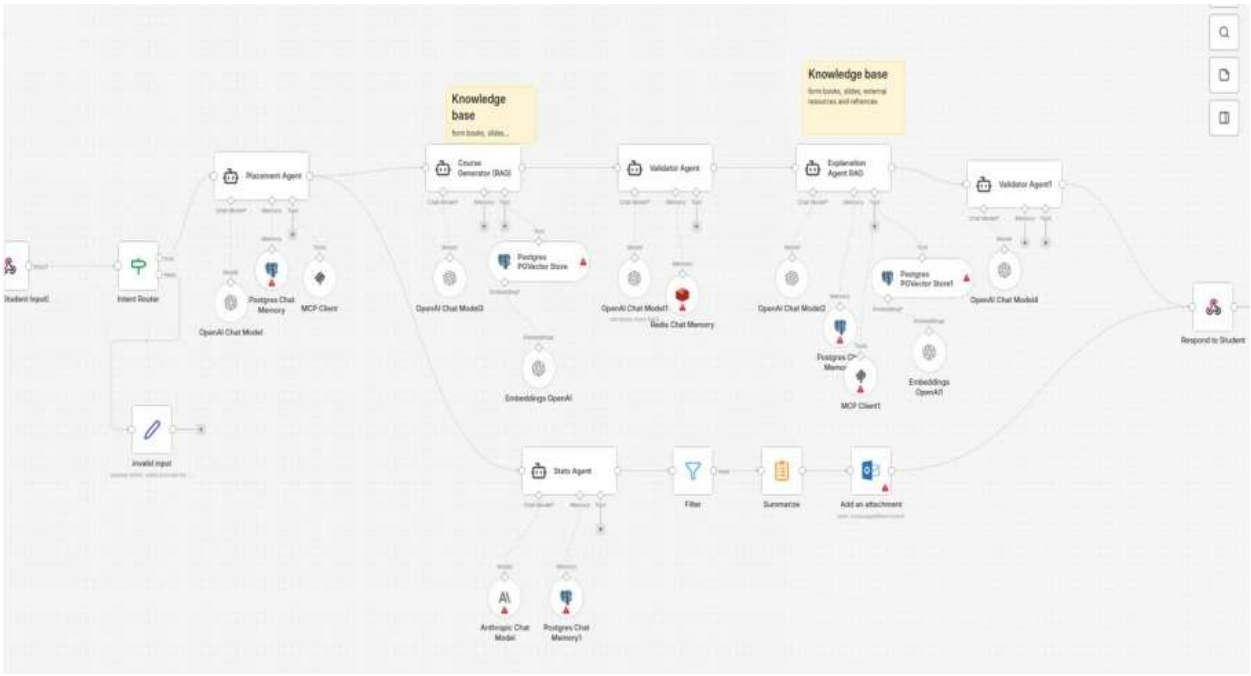


Figure 3.4.2: Agentic workflow illustrating the end-to-end execution pipeline

3.5 UML Diagrams

3.5.1 ER Diagram

Presents the database schema for the students ,learning paths, and interaction log that will be used in the proposed system whereby students are enrolled in a learning path made up of several modules. The interaction between the students and the system is captured in the interaction_log entity that is used in analyzing student learning behaviors and responses from the artificial intelligence system.

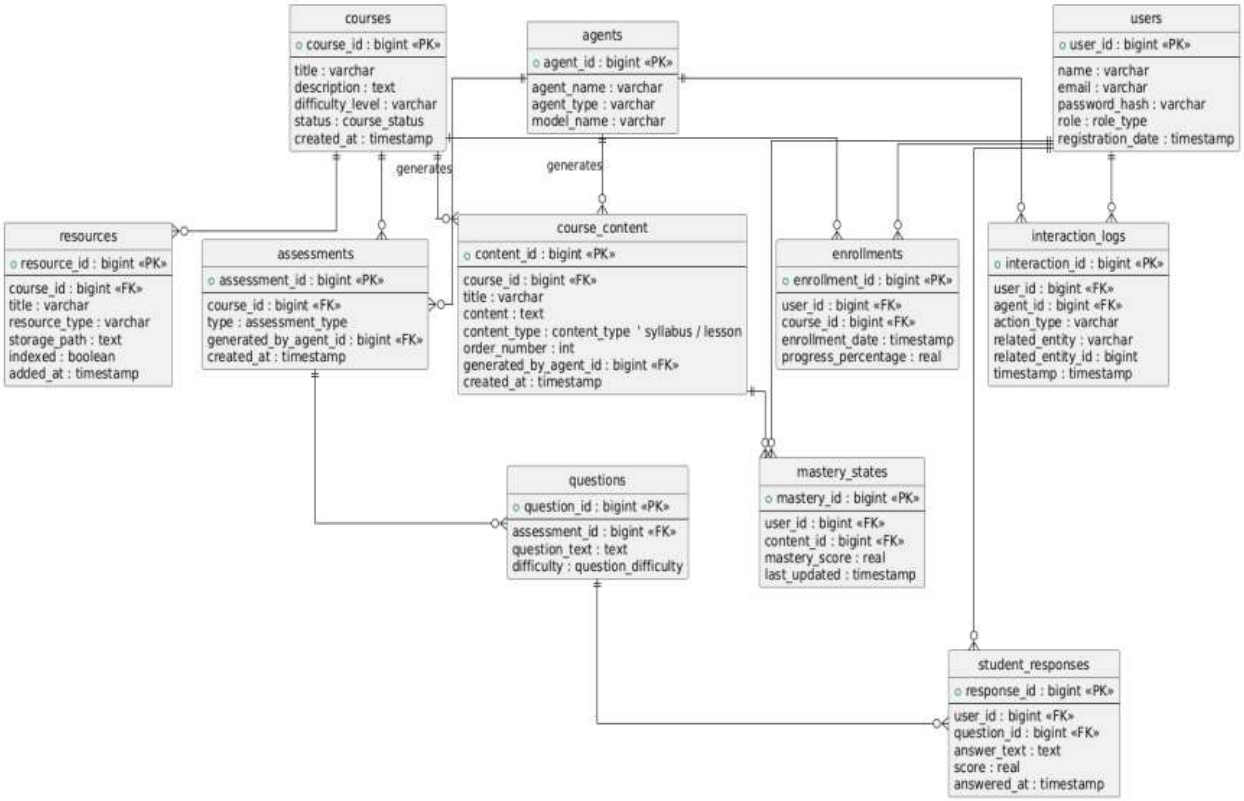


Figure 3.51.: Entity Relationship (ER) Diagram of the Proposed AI Learning System

3.5.2 Sequence diagram

Flow of students' queries from inception, processing, and responses. When a query comes from a student, the Web Interface forwards the query to the MCP Server. The MCP looks into the student's Mastery Profile, as collected by the Analytics Agent, for the tone and difficulty level required. Then the query goes to the RAG Agent that will extract the verifiable facts from the Vector Database. The Explanation Agent provides the final response, which is customized. The response goes back to the student who queried in the first place. The Analytics Agent records the entire transaction to help keep up-to-date the DKT Mastery Score and Risk Score of the student.

Login and Registration

Below is the schematic diagram showing the process of login and registration.

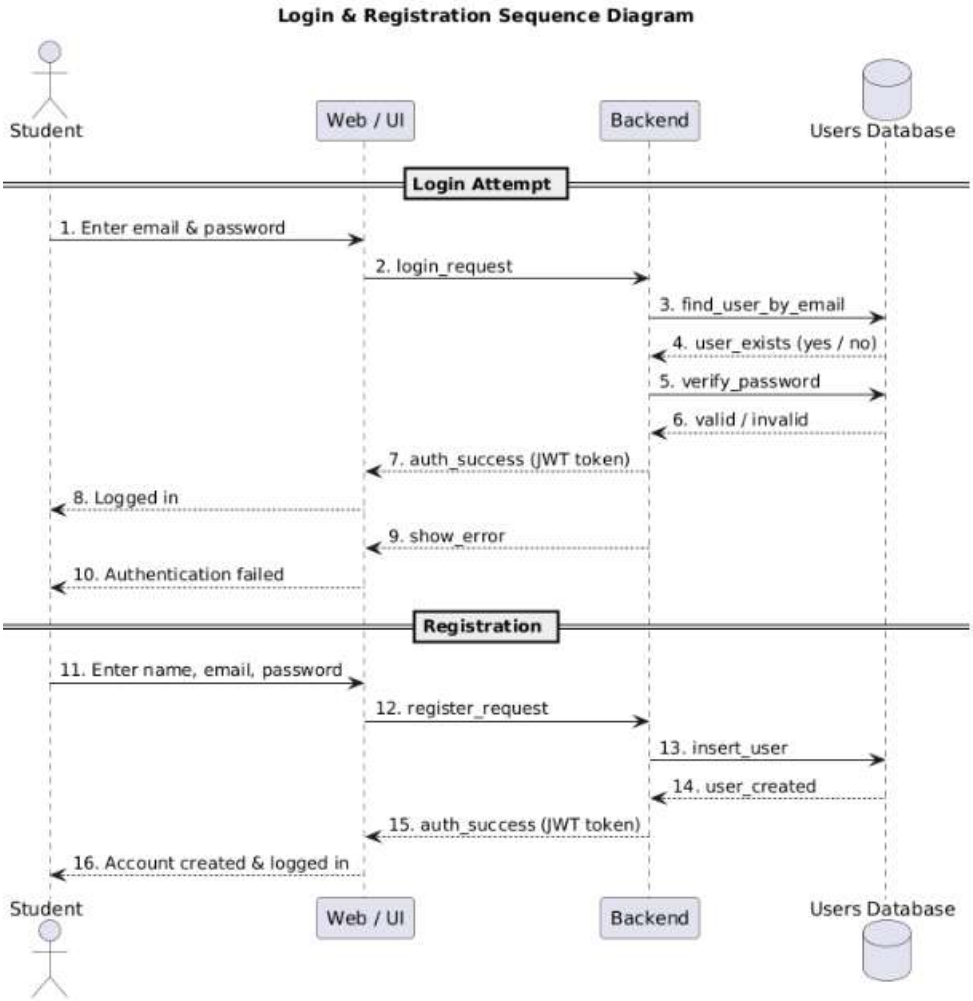


Figure 3.52. Login and Registration Sequence Diagram

Phase 1 – Placement Test

The interaction flow during the placement test process is illustrated above, which involves starting the course by the student, followed by conducting the placement test and evaluating the outcomes to set the student's proficiency level.

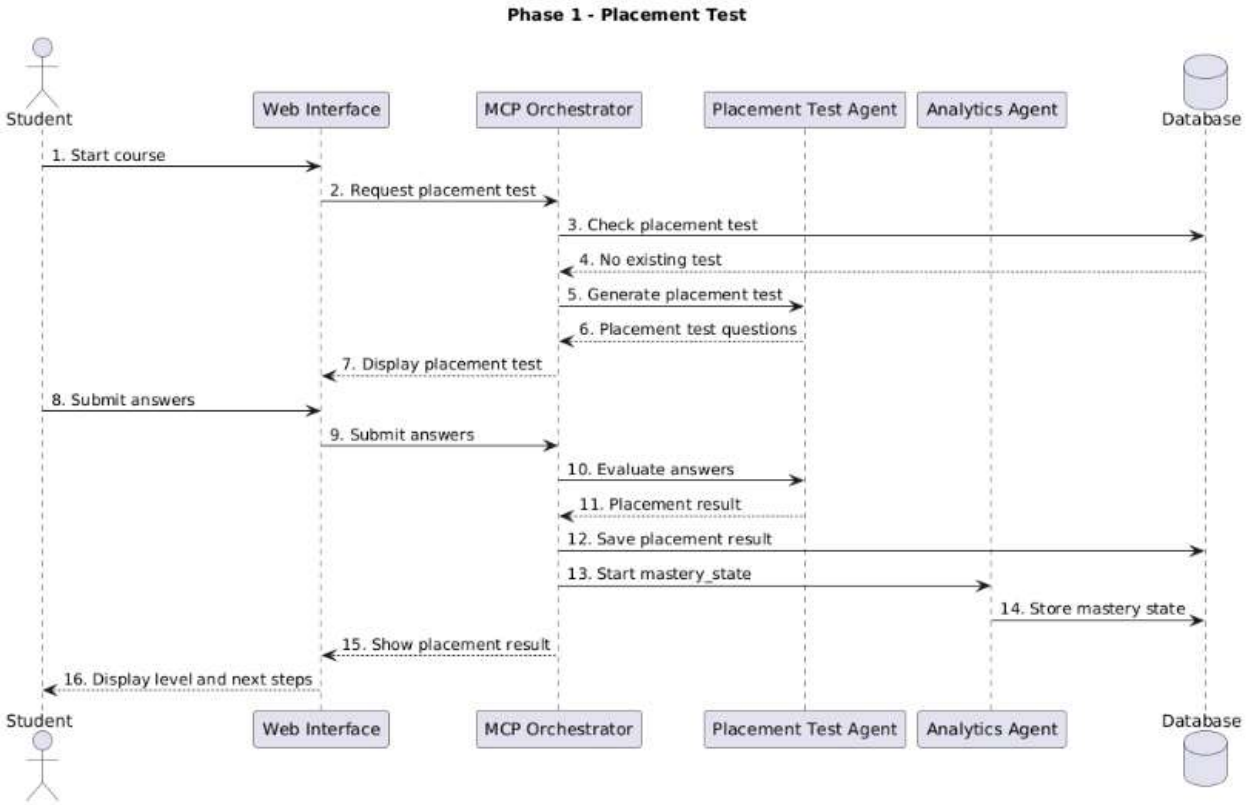


Figure 3.52. Placement Test Sequence Diagram

Phase 2 – Personalized Syllabus & Lesson Generation

This diagram describes how the system generates a personalized syllabus and lesson content based on the student profile and mastery state, including content validation and delivery to the student.

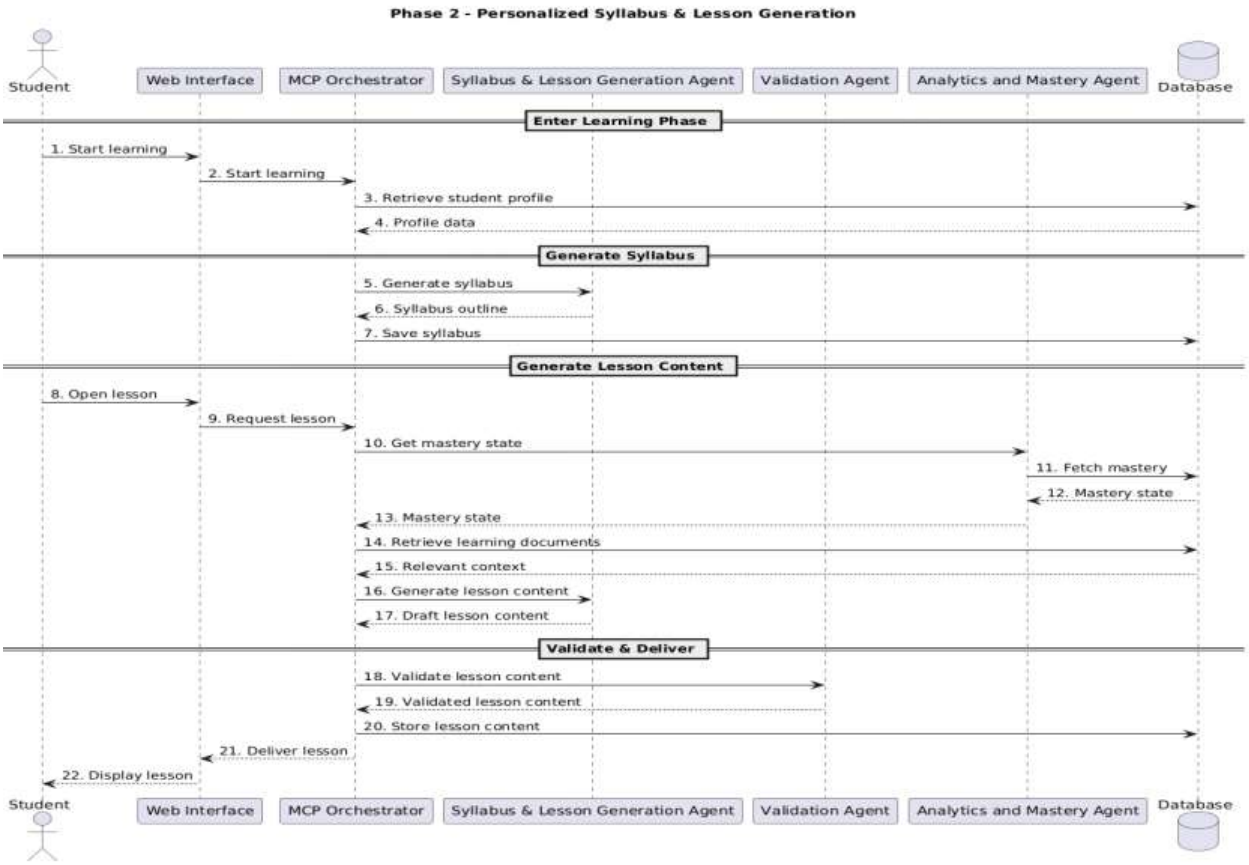


Figure 3.52. Personalized Syllabus and Lesson Generation Sequence Diagram

Phase 3 – In Lesson Question Answering & Explanation

This diagram illustrates the flow of a student's question during a lesson, including context retrieval using the RAG Agent, explanation generation, validation, and mastery state update.

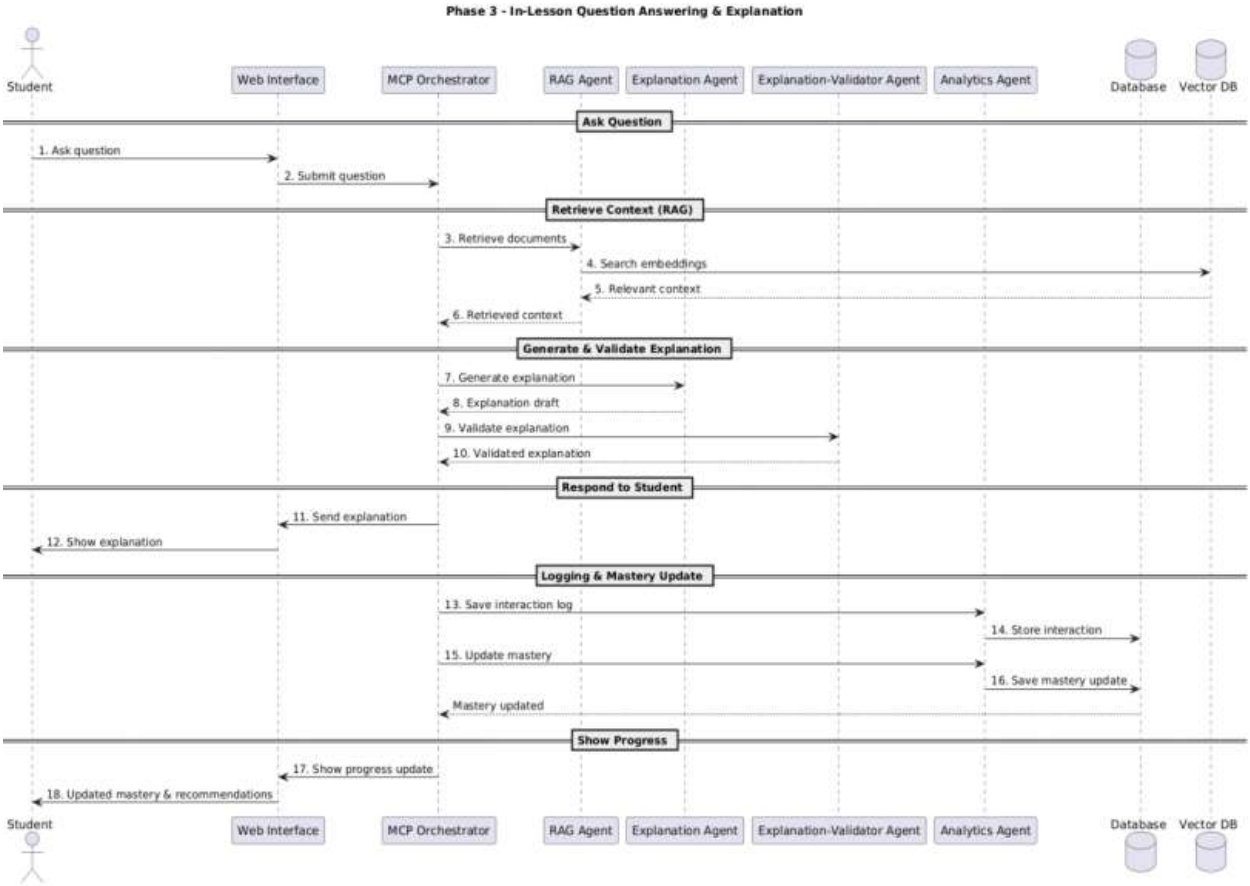


Figure 3.52. In-Lesson Question Answering and Explanation Sequence Diagram

Phase 4 – Assessment & Evaluation

This diagram presents the exam generation, validation, submission, and evaluation process, showing how results and analytics are stored to support adaptive assessment and feedback.

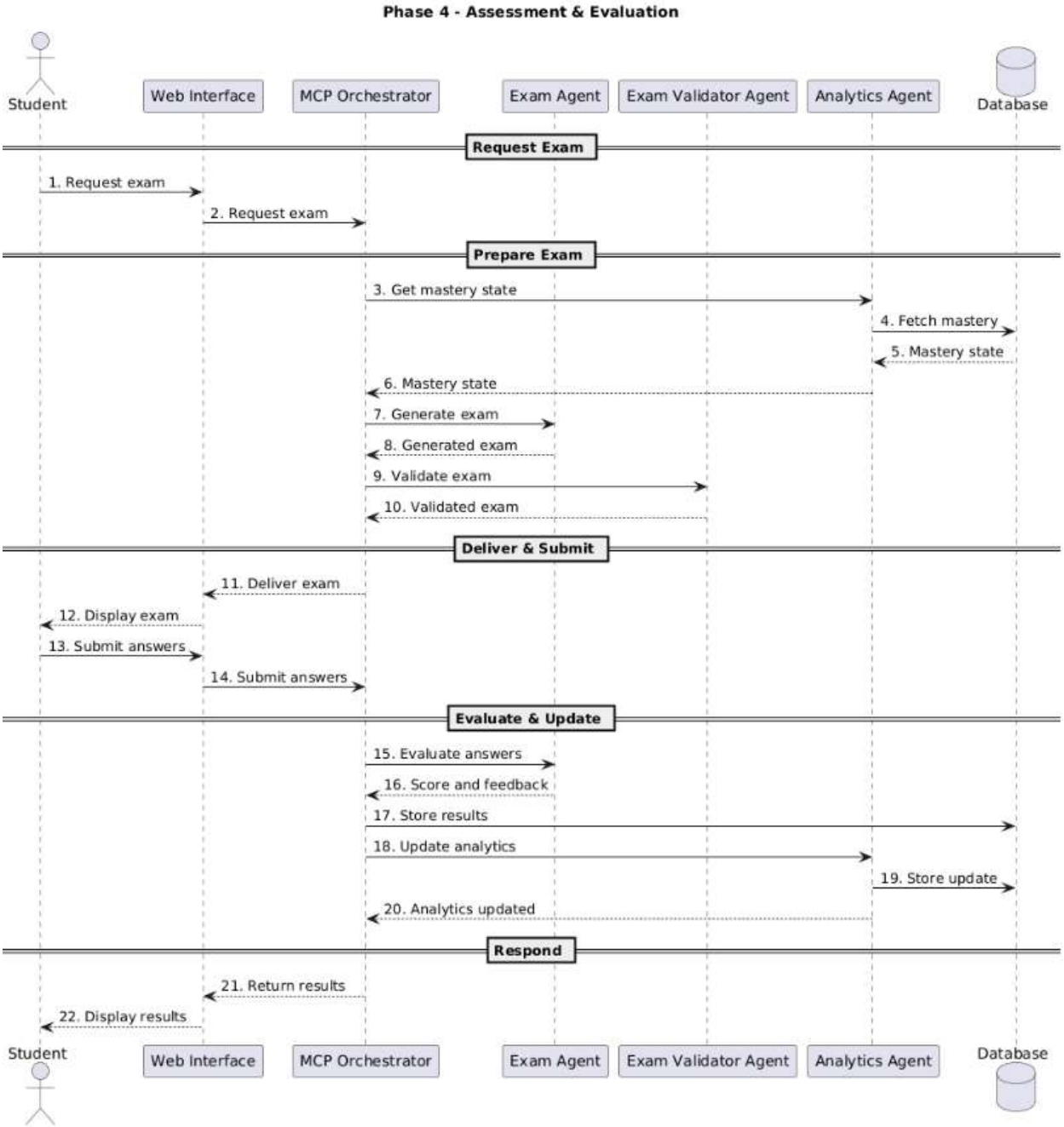


Figure 3.52.Assessment and Evaluation Sequence Diagram

3.5.3 Use Case Diagram

Use case diagram illustrating the main student interactions with the learning system, including authentication, placement assessment, personalized syllabus access, lesson exploration, assessment activities, and performance review.

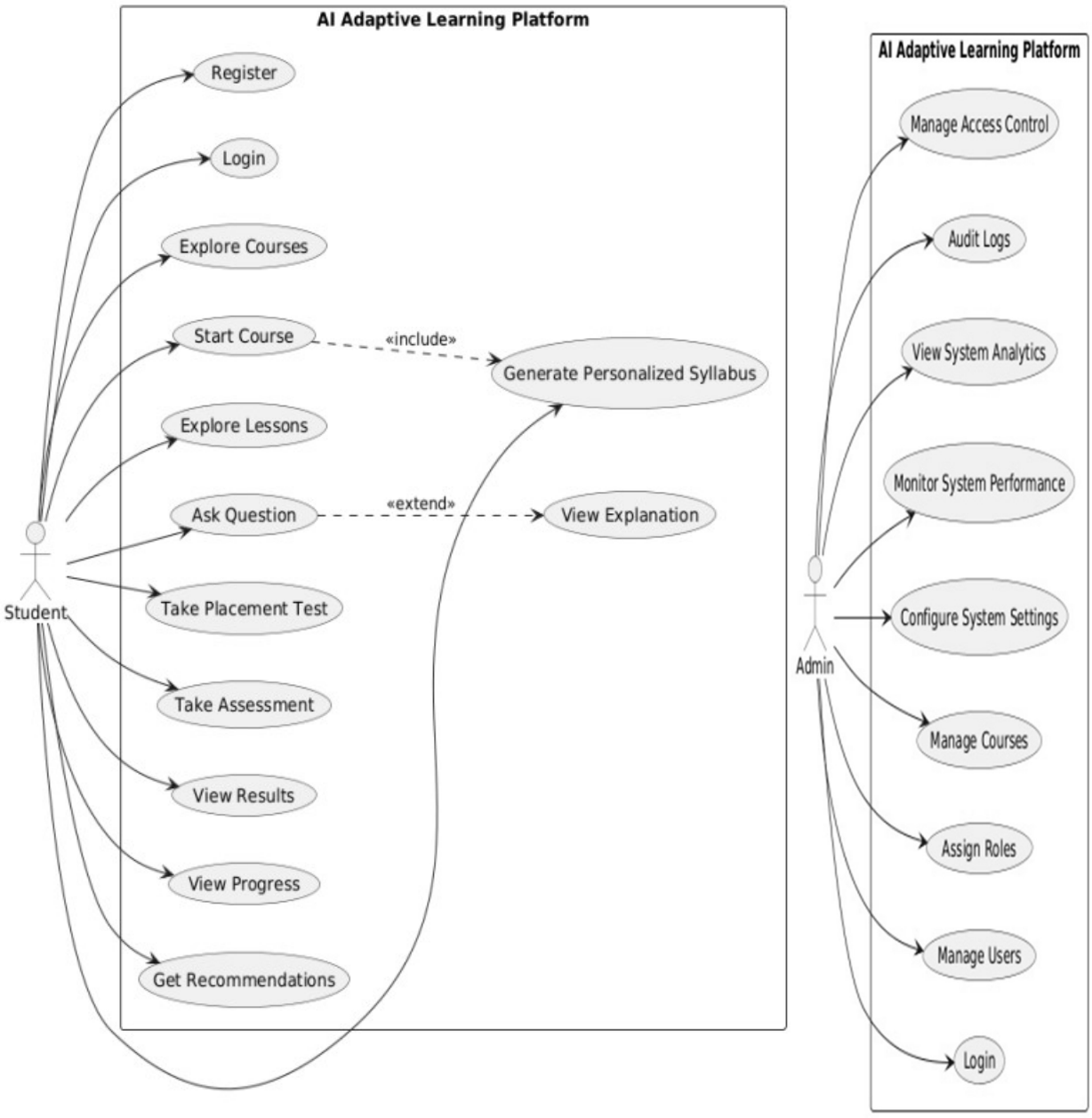
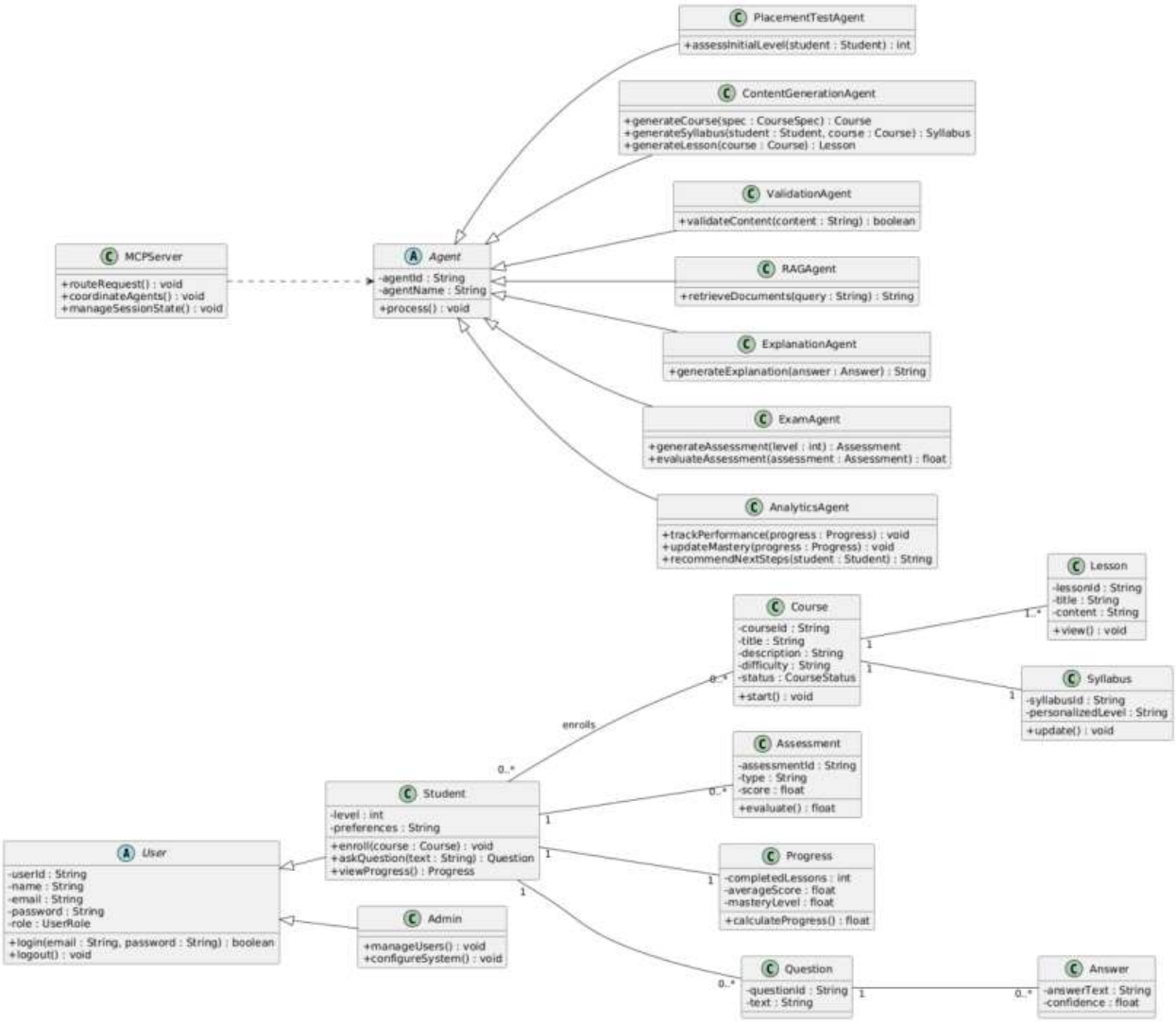


Figure 3.5.3: Use Case Diagram of the AI Adaptive Learning Platform

3.5.4 Class Diagram

Central multi-agent orchestrator coordinates specialized agents to dynamically generate personalized syllabi, learning lessons, assessments, and performance reports based on student interactions



Chapter 4

Implementation

The process of implementation of the system is discussed in this chapter, including choices made, challenges encountered, and obstacles overcome during the process of implementing this system from theoretical ideas to actual practical solutions. The implementation process entailed complex areas such as multi-agent systems, adaptive testing, and prompt engineering. This process did not always go as easily as architecture schemes predicted.

The technology stack used included Python and FastAPI for the backend, React for the frontend, PostgreSQL database via Supabase platform, and Groq's LLM APIs. The first major decision related to the development of this system was using Python-based agents located within one application process instead of creating separate microservices for them. In this way, simplicity was combined with high flexibility necessary for the further development and testing of individual agents.

It should be noted right away that SmartLearner is not a set curriculum developed once and then reused. Each student chooses his or her own learning path: either Python for everybody or deep learning. Content and testing tasks depend on the chosen learning track and are based on the analysis of the following data: the results of placement tests (student's skill level, weak points, and strong points), the results of lesson quizzes, the progress of completing a lesson, and interaction with the analytics page.

4.1 System Implementation Overview

Ultimately, SmartLearner is a combination of special-purpose agents, each of which deals with its own task, and all of which are coordinated via an orchestration layer. Our vision here was simple: don't create one mega-AI solution that tries to do everything; rather, build small modules each focused on doing its job (generating placement questions, developing syllabuses, creating lessons, answering questions from students, and others).

As for the front-end of the system, we used React, TypeScript, and Vite technologies to provide students with a dynamic environment that changes based on their progression through the lessons and quizzes. Data-wise, we decided to use PostgreSQL for persistence and JWT-based keys scoped by the user because at one point it turned out that having multiple users on one browser may make them see each other's information.

We used two separate keys for our Groq API calls — one for generators, where more powerful LLM models are required, and another for the validators to ensure faster processing. Also, to avoid making a free-form LLM generate content without checking it against existing course material, we created a RAG pipeline.

The main services that make up the platform are:

- Placement Generator and Validator
- Syllabus Generator and Validator
- Lesson Generator and Validator
- Q&A Generator and Validator
- Exam Generator and Validator
- Assessment Quiz Service
- Analytics Agent
- RAG Retrieval Service
- Orchestrator Service

Docker and Docker Compose support was added later to standardize the deployment environment. Without it, getting everyone on the team running the same configuration had been a constant source of friction.

4.2 Orchestrator Service

But if there is one element which ties everything together, it is the Orchestrator. All the requests from the frontend get handled by the Orchestrator, all agents are triggered by the Orchestrator, and all the data flows through the Orchestrator in its way towards the database or the user. The Orchestrator was implemented as a Python class, referred to as OrchestratorService, and was injected into FastAPI route handlers using the mechanism of dependency injection. The architecture of the system was designed based on the hub-and-spoke idea, where the Orchestrator works as the hub and every agent operates as a spoke connected to the hub. No communication occurs between the agents. As it appeared later, this architecture was as useful for us as it was convenient because in case some agent was unable to complete its task, or had to be replaced with another, this wouldn't affect anything apart from logging which took place at the level of the Orchestrator, thus giving a full overview of what went wrong in the process. The main pattern of workflow in the system can be expressed as follows: Generate → Validate → Log → Persist. The generated content is checked by the validator and the result, whether the operation was successful or not, is saved to the agent_runs table prior to the response being sent to the student. This logging proved very helpful in finding the cause of failures and prompt misbehaviour in the process of development.

Placement Workflow Coordination

At the start of each placement session, the Orchestrator initializes the session in the database and triggers the Placement Generator to create one question at a time per rubric slot, as opposed to getting five questions in bulk. This modification was made since it was observed that sets of five

questions generated together always used the same templates. Generating questions individually, with scenario hints and shuffling RAG context by rubric slots, generated diverse outputs.

Following every response received from the learner, the session state is updated in the Orchestrator. A score of four or above out of five results in the learner moving forward with the test, while anything less than that ends the placement process and allows storing of the weak and strong concepts for future syllabus creation.

Syllabus Workflow Coordination

After the placement process, syllabus creation begins based on the placement data collected, weak and strong concepts, and learner's chosen learning track. Following syllabus creation, there will be validation to ensure that the syllabus is valid in terms of topic name, chapter sequence, lesson structure, and minimum sub-lessons count.

Lesson and Remediation Coordination

The Orchestrator verifies that all required prerequisites are accomplished before starting a new lesson by the learner. If a new lesson content needs to be created, the Orchestrator invokes the Lesson Generator, ensures that the lesson is created correctly, and stores the content in the database.

When a learner fails the end-of-the-lesson quiz for less than four answers out of five, remediation mode kicks in. The initial lesson is divided into two or more sub-lessons, depending on how difficult it was. All sub-lessons are created anew, explaining the subject in detail, focusing on what the learner got wrong. The final quiz consists of questions from the last sub-lesson.

Q&A and Exam Coordination

Firstly, the requests will pass through a process of sanitization as well as a process of determining their relevance. In the event that the question appears to have no bearing on the content taught, the request will be automatically discarded. The remaining questions will be handled by choosing an appropriate explanatory approach and then coming up with answers by finding relevant information using the RAG method. In regard to creating exams, questions dealing with areas in which the student finds difficulty will take priority, while the others will just be brief and de-duped to avoid having duplicate questions posed to the student.

Analytics Coordination

Analytics are calculated deterministically in Python: proficiency levels, progress indicators, learning speed, risk factors. In this case, the task of the LLM is restricted to creating a story overlay – personal summary, motivational text, and follow-up action. It is explicitly prohibited to use the LLM for generating any number-based data, as this was proved crucial due to the unpredictability of LLM-generated numeric data.

4.3 Placement Workflow Implementation

The placement procedure of the students in the right classes depends on the rubrics that change for each track. For example, the rubric for the Python track includes the concepts of the Python for Everybody textbook, while the rubric for the Deep Learning track refers to the Deep Learning textbook content. Once the session has started, the Orchestrator determines the session state and calls for five questions about the selected difficulty level. These questions are generated separately by sending calls to the LLM, where the rubric concept serves as the starting point.

Each prompt has some scenario hints added to ensure variety and shuffle RAG chunks per slot to prevent repetition between sessions. The Placement Validator validates each of the questions according to their structure, adherence to the rubric, answer truthfulness, and concepts order.

The session state is kept in the database with the current level number, question number, scores, weak concepts and strong concepts being saved each time after an answer. Reaching the score of four or higher brings the student to the next level. Otherwise, the process ends and the last level reached as well as the concept profile are saved.

4.4 Syllabus Generation Implementation

The process of syllabus creation involves using the placement test results to generate a personalized syllabus. We make the weak topics have extended content, and the strong topics will be minimized without being completely stripped away. The hierarchy of chapters and lessons in the generated syllabus will follow that of the source material (Python for Everybody and Deep Learning).

This component has some stringent requirements. It requires the names of topics to correspond strictly to an allow list, a lesson title to not be a mere copy of the topic name, a minimum of four lessons per chapter, and to keep the original order of chapters. At first, our initial prompts failed multiple times because of generating flat arrays of lessons or having the title prefix "Practical". This resulted in the prompt structure outlined in Section 4.10.

After validating a syllabus, we write it to PostgreSQL as a course entity with lesson rows. These will be sorted and returned in groups of units to the frontend. In case of failure in creating a syllabus with LLMs, we fall back to locally-generated content.

4.5 Lesson Generation and Remediation

Lessons are created as needed the very first time they're opened by the student and cached afterward. The Lesson Generator creates the complete markdown lesson – objectives, explanations, solved examples, coding instructions, practice problems, pitfalls, and a summary – based on RAG context and the student's placement level.

The Lesson Validator ensures source consistency by mandating that the lesson be sourced from Python for Everybody or the Deep Learning text, not fall outside the current topic, and have the correct markdown formatting. In the event Groq fails, a markdown template is provided instead of leaving the student with an empty screen.

Remediation occurs if a student gets less than four of the five questions right during the first pass on a root lesson. The lesson will be split into two to four sub-lessons and will automatically regenerate with clearer explanations, easier-to-understand examples, and guided practice focusing on the error committed on the quiz. A student will be allowed three attempts at retaking a failed quiz, after which the lesson will become uneditable; a failure on a sub-lesson will lead to a new set of questions being generated.

4.6 RAG Pipeline Implementation

The entire purpose of the RAG pipeline is to reduce hallucination. Instead of generating educational content from scratch, every generation request is based on educational content that has been generated offline: chunks of content scraped from Python for Everybody for the Python track and selected textbook excerpts for the Deep Learning track. All those chunks are stored offline in a chunk index file.

In the process of generating the response, the RAG service will use a lexical token overlap score to find the most relevant chunks, select them, and include in the agent's prompt before calling the LLM model. This approach is more efficient than embeddings, especially when used on every request, and guarantees that all content is based on verified sources. If no chunk index is available, we'll have a handful of hardcoded prompts that ensure our agents will function in development mode.

In case of Q&A, there is also a relevancy gate, which allows us to reject any question irrelevant to the educational content if we think the student is asking the wrong kind of questions. It helps us significantly reduce off-topic responses.

4.7 Q&A Agent Implementation

The Q&A agent tailors its explanations depending on the student's proficiency and the nature of the problem posed. There are three explanation modes – supportive, balanced, and succinctly technical – which are automatically chosen depending on the category of the question posed; the question categories include conceptual, how-to, compare-and-contrast, and debugging problems among others.

Each reply is put through sanitation, retrieval, generation, and validation processes prior to delivery to the student. Validation of responses entails checking their source grounding, ensuring that Python 2 material does not appear in Python 3 classes, and including a follow-up question. All responses are delivered in structured JSON format complete with markdown and follow-up questions.

4.8 Exam and Assessment Implementation

4.8.1 Quick Lesson Assessment

Each lesson concludes with a five-item multiple-choice test based on the lesson itself with the variation seed in order to avoid repetitions. In order to pass, students have to answer correctly four out of five questions, getting no more than three attempts for this test. On each attempt, a new seed will be used and questions repeating the previous one in any way will not be accepted. For split lessons, the test will appear at the end of the last sub-lesson.

4.8.2 Full Exam Generation

Exams consisting of three, five, or ten questions will be created by Exam Generator taking into account such aspects as the difficulty balance, preference to weak subjects, duplication avoidance and Exam Validator will check every item according to the specified criteria. In case the generation failed after multiple attempts, the remaining number of questions will be filled with the pre-defined templates.

4.9 Analytics Implementation

In analytics, there is a clear distinction between deterministic computations which are performed on the Python side and narratives themselves which are generated with the help of LLM. The reason for splitting into two sides was that the direct usage of LLM for numeric calculations was ineffective as it provided either erroneous or unreasonable results.

The Analytics Agent produces summaries, motivation advice, as well as recommendations about next steps based on numerical results. No modification of any values calculated on the Python side is allowed by the Analytics Agent's prompt. Logs are written by the agent itself after finishing an agent run as usual.

4.10 Agent Prompt Engineering

Prompt engineering ended up being one of the most time-consuming parts of the implementation. Early prompts across almost every agent produced problems: hallucinated content, repeated question stems, flat structures that failed validation, outputs in the wrong format, or responses that drifted away from the course material. Getting prompts to reliably produce structured, grounded, validator-compatible output required multiple rounds of testing and rejection.

The final approach uses highly constrained prompts with JSON-only output requirements, explicit formatting rules, rubric alignment instructions, RAG grounding, and structures designed with the validator's expectations in mind. What follows are the adopted and rejected prompts for each agent, with notes on why the rejected versions failed.

4.10.1 Placement Generator Agent Prompt

Final Adopted Prompt:

MCQ placement test generator based on the source material of the selected track. In the user message, you will get a rubric_concept. Generate EXACTLY ONE question on THAT concept ONLY. DO NOT include any other concept or any syntax/ideas from another chapter. Scope for each concept at each level dynamically generated in the prompt. Stick strictly to that scope. Beginner Level - Avoid anything related to framework and system design, unless it comes in the scope. JSON Format: question(string), options(array of exactly four strings, with no repetition between questions), correct_answer(must be exactly as one of your options), concept(same as rubric_concept). Aim at testing concepts, not memorization/trivia. Ensure each of the five questions uses a UNIQUE scenario with varied stories, code snippets, and distractors.

Rejected Prompt:

MCQ Generator for Placement. Generate ALL 5 MCQs in a SINGLE JSON response under the key "questions".

Rejection reason: Repetitive question patterns across all slots, poor rubric correlation, and substantial deviation from the intended concept while creating all 5 questions in a single request.

4.10.2 Syllabus Generator Agent Prompt

Final Adopted Prompt (excerpt):

Syllabus generator for track 'python'. Generate a personalized syllabus starting ONLY from the student's placement level. Do NOT include content from lower levels. Return JSON with top-level key "units" (array).

Do NOT return flat lessons[] only. Each sub-lesson MUST have: topic (exact allowlist match), lesson_title (descriptive, not equal to topic), description, learning_objectives, rubric_concept, chapter_ref. Minimum 4 sub-lessons per chapter.

Rejected Prompt:

Generate a personalized Python syllabus as JSON with key "lessons" (flat array). Each lesson needs: topic, title, description. Prefix every lesson_title with "Practical".

Rejection reason: Flat structure failed the validator's unit hierarchy requirement. Prefixed titles were flagged as duplicates. No chapter ordering or sub-lesson count enforcement.

4.10.3 Lesson Generator Prompt

Final Adopted Prompt (excerpt):

Lesson generator for Python for Everybody. Return JSON with a single key "markdown". Write a substantive lesson (600–1200 words) using markdown headings, at least two code blocks, worked examples, practice sections, pitfalls, and summaries.

Rejected Prompt:

Write a short lesson summary in bullet points. Keep it under 200 words.

Rejection reason: Content too shallow for actual learning. No code examples, no practical exercises. Failed the lesson validator's grounding and content checks.

4.10.4 Q&A Generator Prompt

Final Adopted Prompt (excerpt):

You are an adaptive AI tutor assistant. Respond ONLY with valid JSON. Use the provided RAG context as primary evidence. Adapt teaching style using ADAPT_MODE. The answer field MUST contain the course source name.

Rejected Prompt:

You are a helpful Python tutor. Answer the student's question clearly. Return plain text only.

Rejection reason: No adaptive behavior, no structured output, and no grounding requirement. Produced inconsistent formats that could not be reliably parsed or validated.

4.10.5 Analytics Agent Prompt

Final Adopted Prompt:

You are the Analytics Agent for SmartLearner. NEVER change any numeric value. Return only personal_summary, next_action, and motivating_recommendations as JSON.

Rejected Prompt:

Compute mastery scores and risk scores directly using the LLM.

Rejection reason: LLM-computed scores were non-deterministic and unstable across runs. The same student data could produce different mastery values on different calls, making analytics meaningless.

4.11 Challenges Faced and Solutions Found

Implementing SmartLearner brought several practical challenges not envisioned during architecture planning. Below is a list of the most important challenges, along with their respective solutions.

4.11.1 Environmental Differences Between Developers

Challenge:

Developers using different versions of their dependencies would result in workflows that worked for one person but broke for someone else due to inconsistencies between environments. Particularly frustrating was when the team had to deal with changes more often than usual at the start of the project.

Solution:

We started using Docker with Docker Compose in order to provide an equal development environment for everyone. Environment variables, database credentials, and RAG resources were all placed inside containers. Dependency versions were also pin-pointed within requirement files of those not using Docker.

4.11.2 Repetitive Placement Questions Stem

Challenge:

Placement tests tended to generate the same question stem when multiple sessions were conducted. The reason behind that was a consistent ordering of rubrics in the prompt, the retrieval of similar contexts via RAG, and low generation temperatures.

Solution:

Each of these questions got its unique session variation identifier, concept-specific scenario description hints, randomized RAG chunk selection, explicit instructions for anti-repetition in prompts, and sequence checking for rejecting near-duplicates of the previously generated stem.

4.11.3 Invalid Responses From Syllabus Generator and Structure Drift

Challenge:

Syllabus generation always produced lesson structures that consisted only of units[], contained less than 4 lessons in every chapter, had no title other than the topic name, or were prefixed with 'Practical', which caused the validation process to fail. None of these structures qualified for further processing by the program.

Solution:

Syllabus generation prompt underwent an upgrade to specifically define units[] structure, at least 4 sub-lessons per chapter, descriptive titles distinct from topic names, and exact topic matching in allowlist. A fallback local syllabus was also used in case of repeated failures.

4.11.4 Groq Rate Limits and LLM Failures

- **Problem:**
  Groq's rate limits and transient API errors interrupted generation workflows at unpredictable times, sometimes mid-session. Returning hard errors to the student was not acceptable.
- **Solution:**
  The Orchestrator now includes retry loops and deterministic fallback content for every major generation workflow. Students get a usable result even when the LLM is temporarily unavailable, and the degradation is graceful rather than visible.

4.11.5 Agent Hallucinations and Unsubstantiated Content

Problem:

Content from the generated lessons and Q&A might be incorrect or mention concepts not within the scope of the course. Without a validation layer, such content could go straight to the students.

Solution:

Validation becomes a necessary step in the Generator-Validator model. The combination of RAG grounding, restricted scope, citation policy, and deterministic validation significantly mitigated issues with unsubstantiated content.

4.11.6 Multi-Agent Orchestration Latency

Problem:

Agent execution took time. Sessions with multiple iterations of generating and validating were slower, as was syllabus creation with an extensive RAG context.

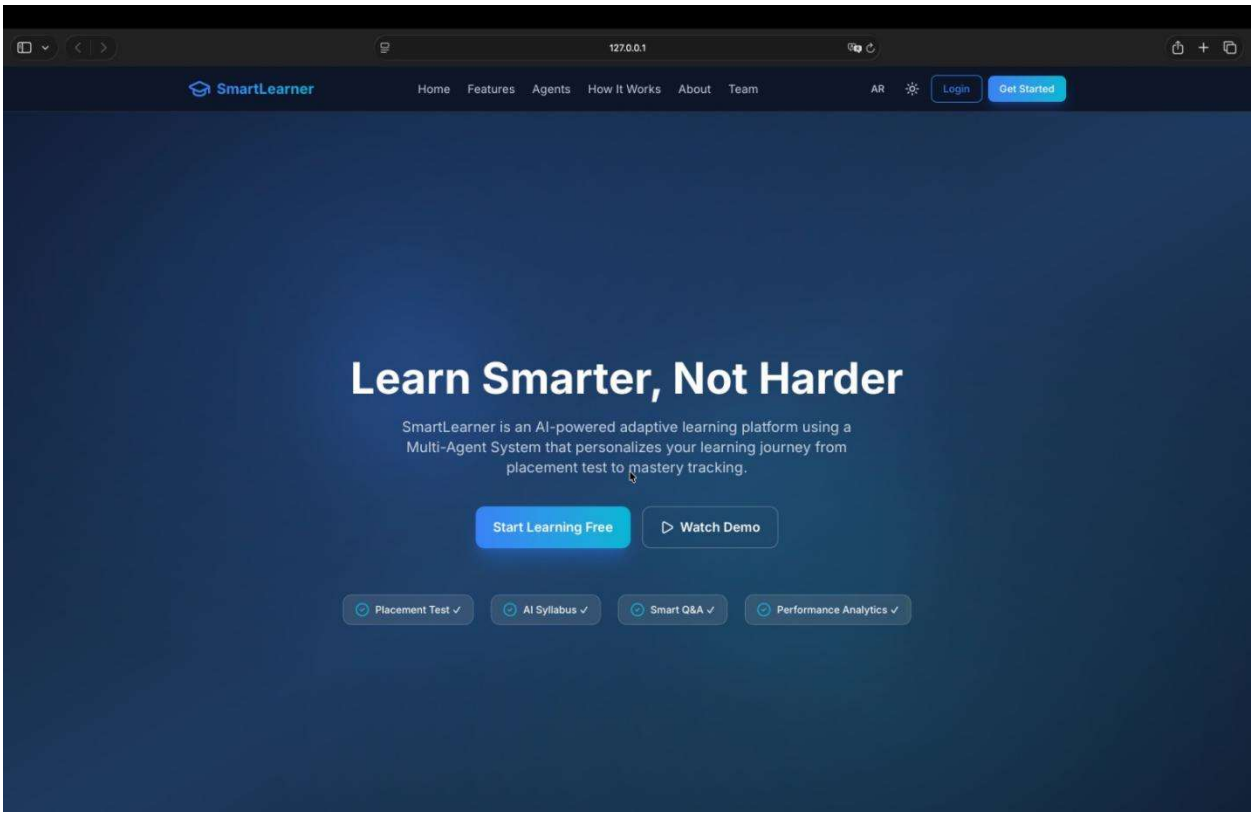
Solution:

Caching of the lesson in the database removed the need for regeneration for the most frequent case. Faster model tier was used for validation and analytics narrative layers, where correctness is of secondary importance to performance. Attempts at retry were kept for cases when the improvement was justified.

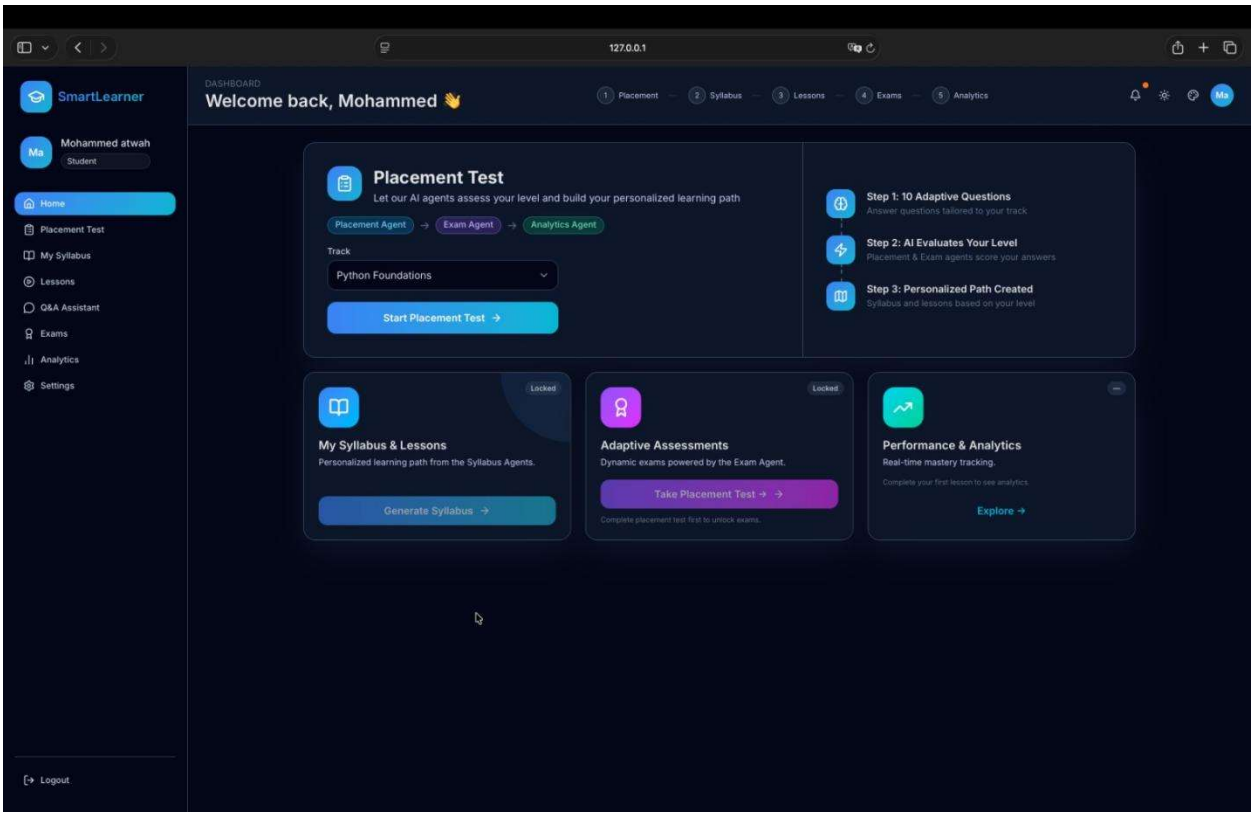
4.12 Summary

The building of SmartLearner was examined in this chapter in terms of architectural choices made during its development, the implementation of components, prompt engineering and other aspects as well as issues and solutions faced by us throughout the implementation. As an end result, we designed a multiagent system with independent agents performing specific actions according to centralized orchestration with validation guaranteed prior to any output reaching the learner.

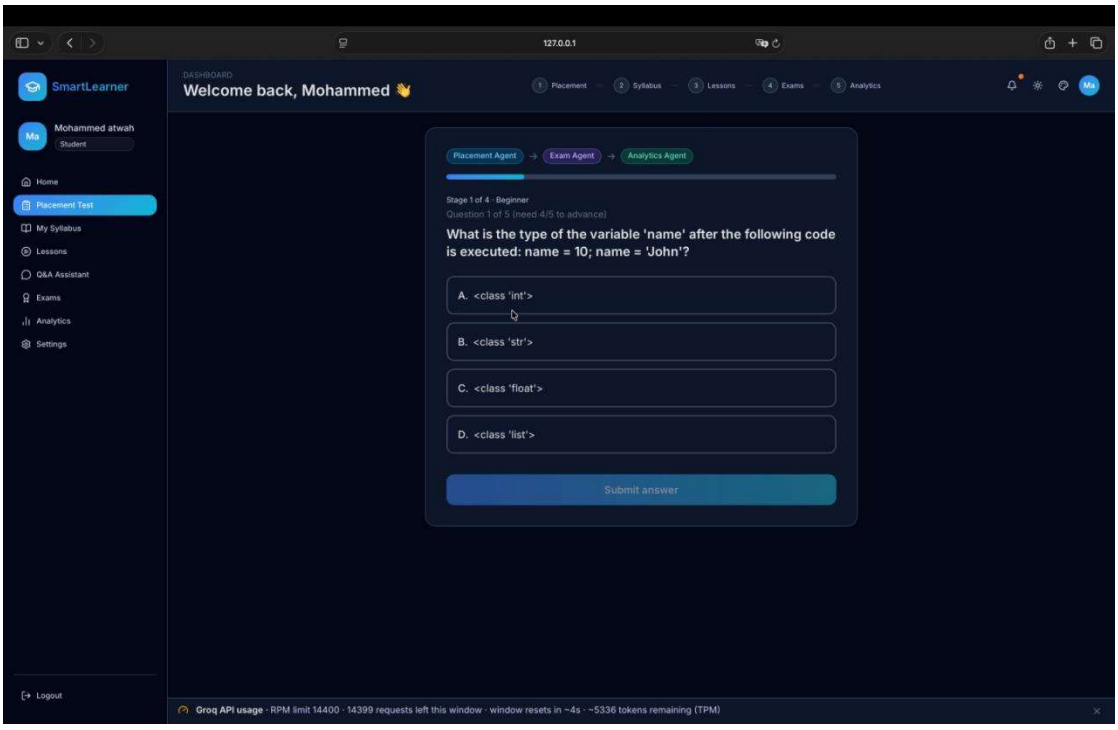
The Generate → Validate → Log → Persist workflow may be considered the most important architectural choice for our work since it offered an opportunity to ensure high quality prompts and traceability. At the same time, fallbacks at each layer helped us ensure that the tool could still be usable in case the APIs were not available. Some of the problems mentioned in Section 4.11, including repetition questions, state leakage between accounts, and the inability to validate, occurred in real life rather than in theory and made us come up with solutions that were effective but not beautiful. We believe that the solutions to these problems will be very helpful when we develop our product further.



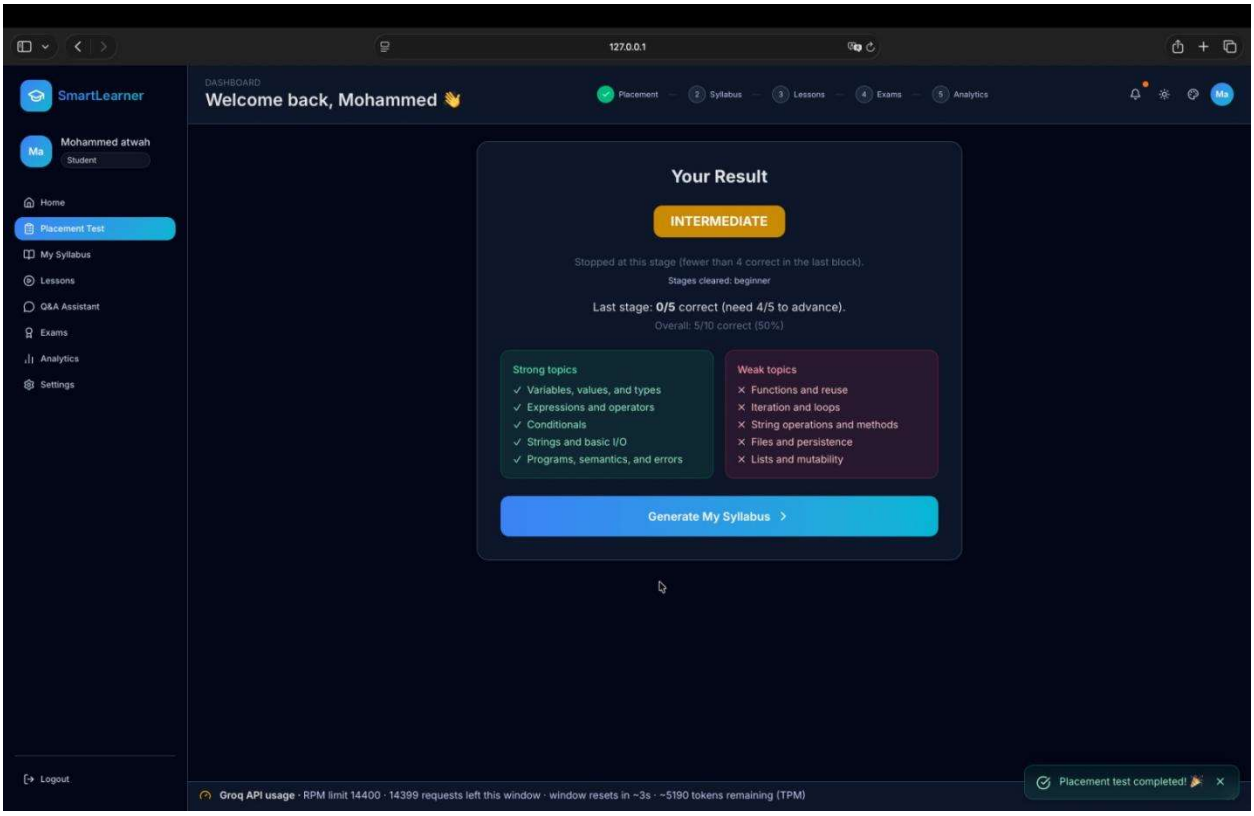
1. Dashboard Page



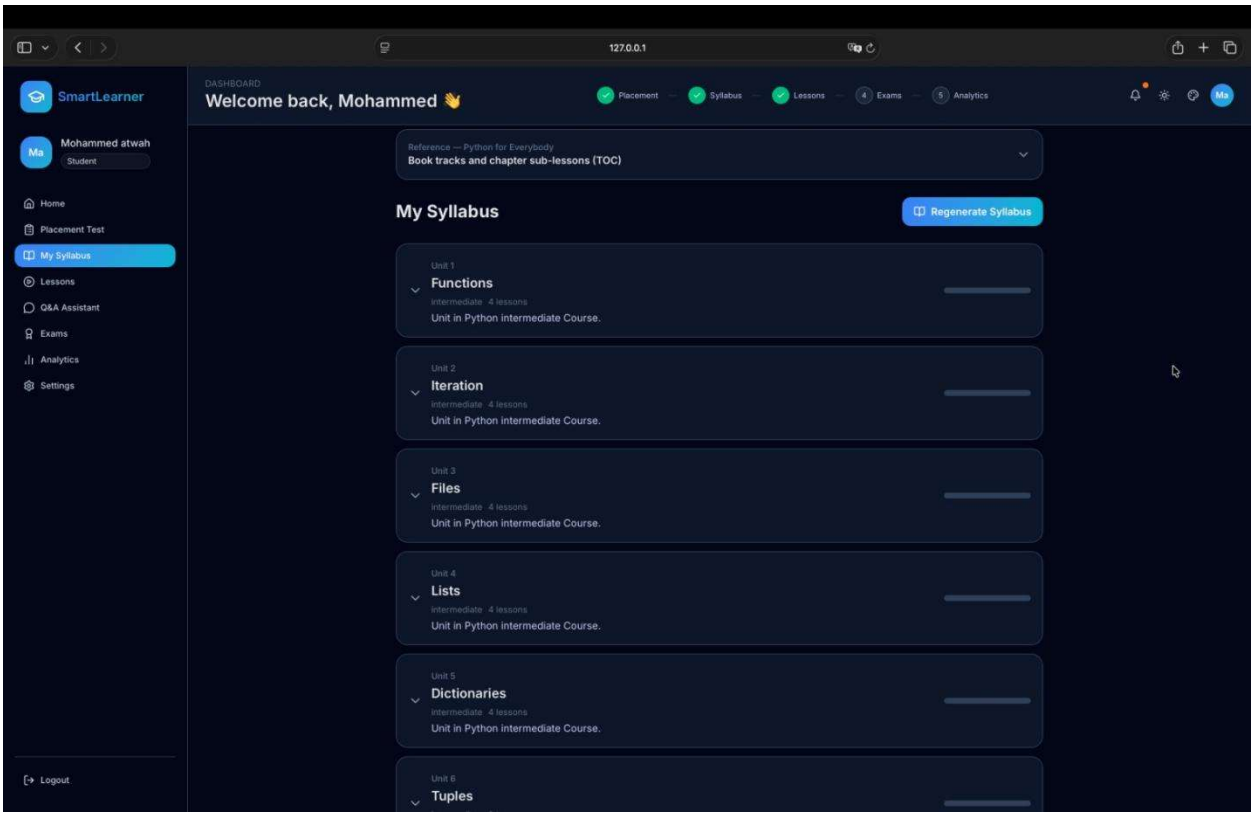
2. Main dashboard showing the student's learning progress and lesson navigation.



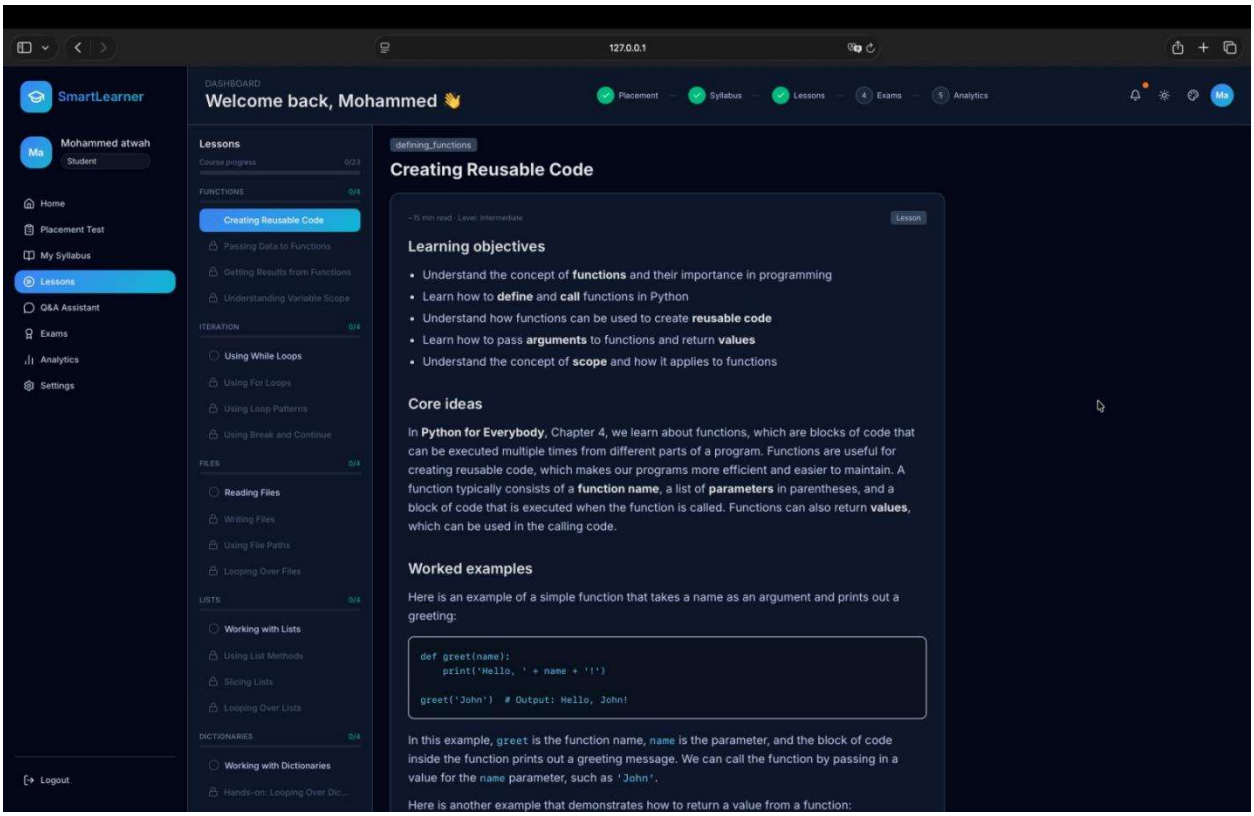
3. Placement Test Question (Adaptive placement test question used to assess the student's skill level.)



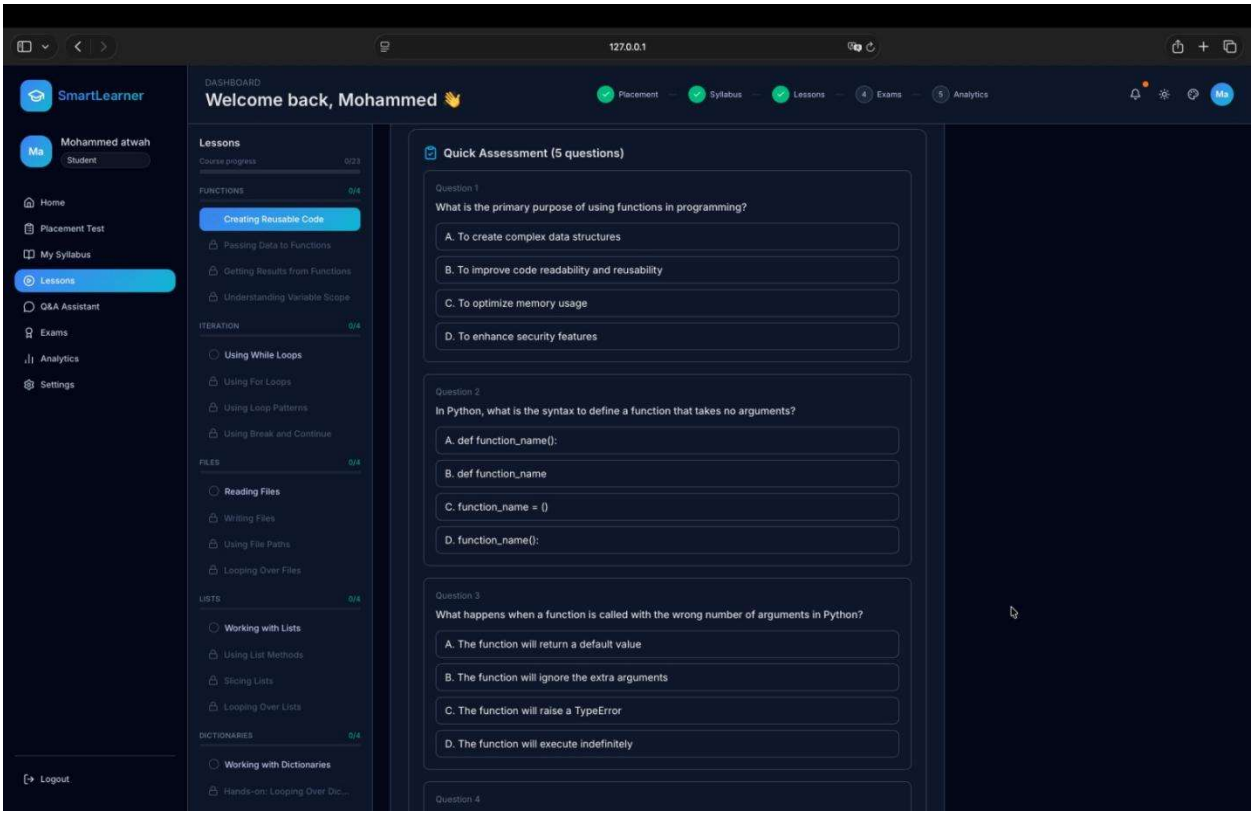
4. Placement Test Result (Final placement test results highlighting strengths and weak areas.)



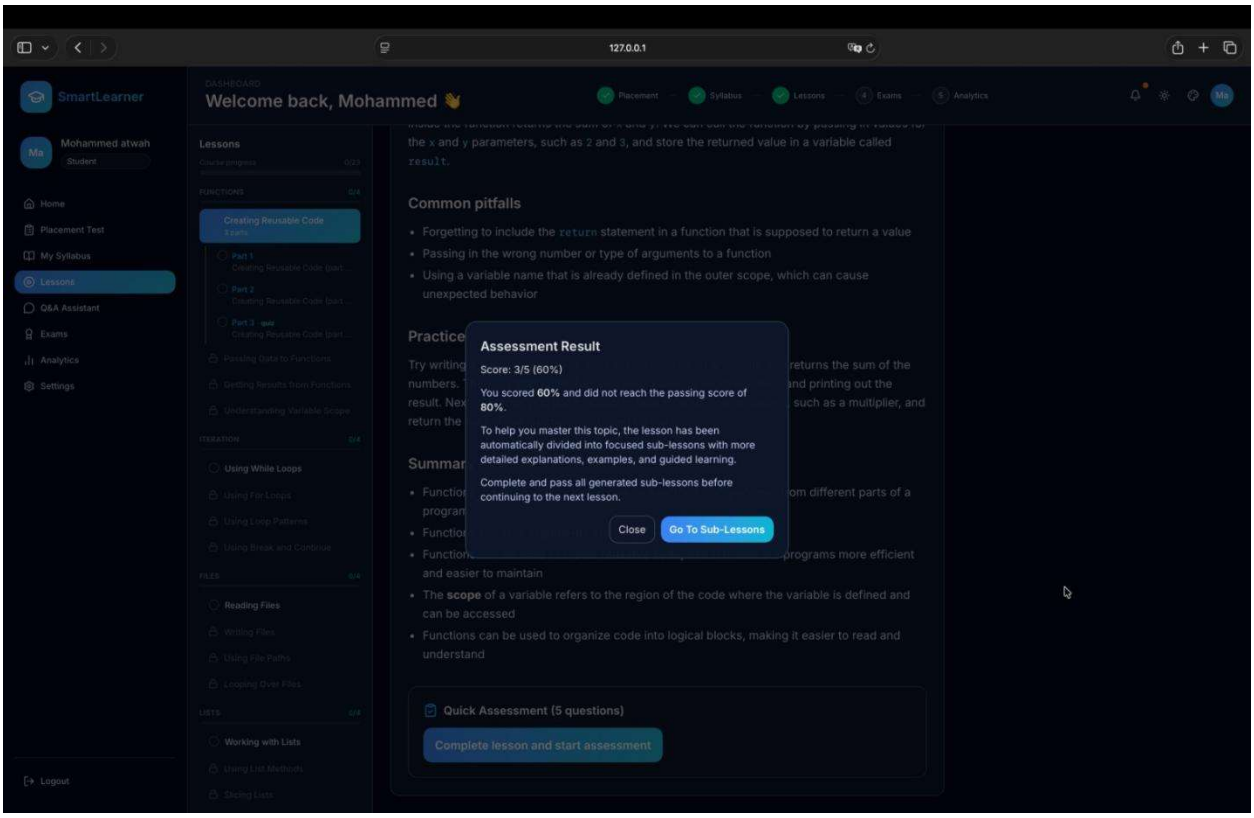
5. My Syllabus Page (Personalized syllabus generated based on the student's assessed level.)



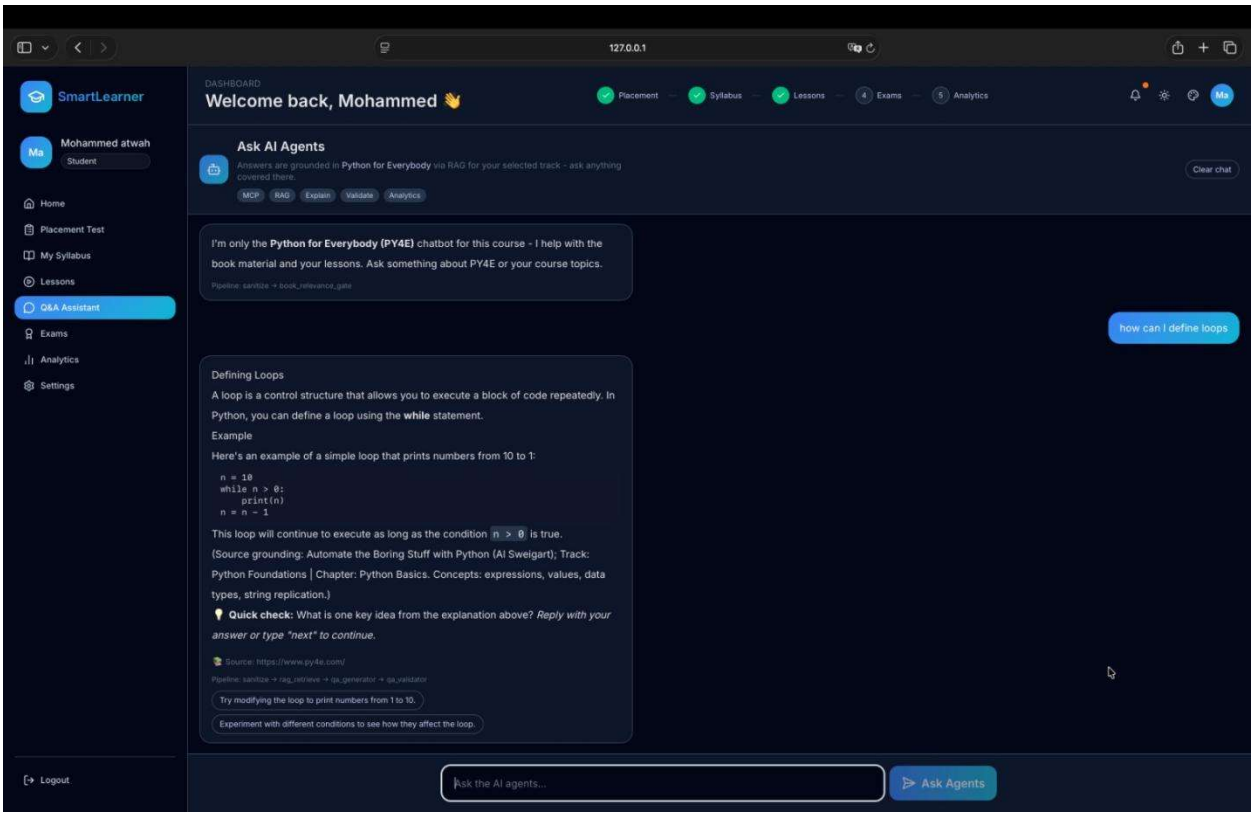
6. Lesson Content Page



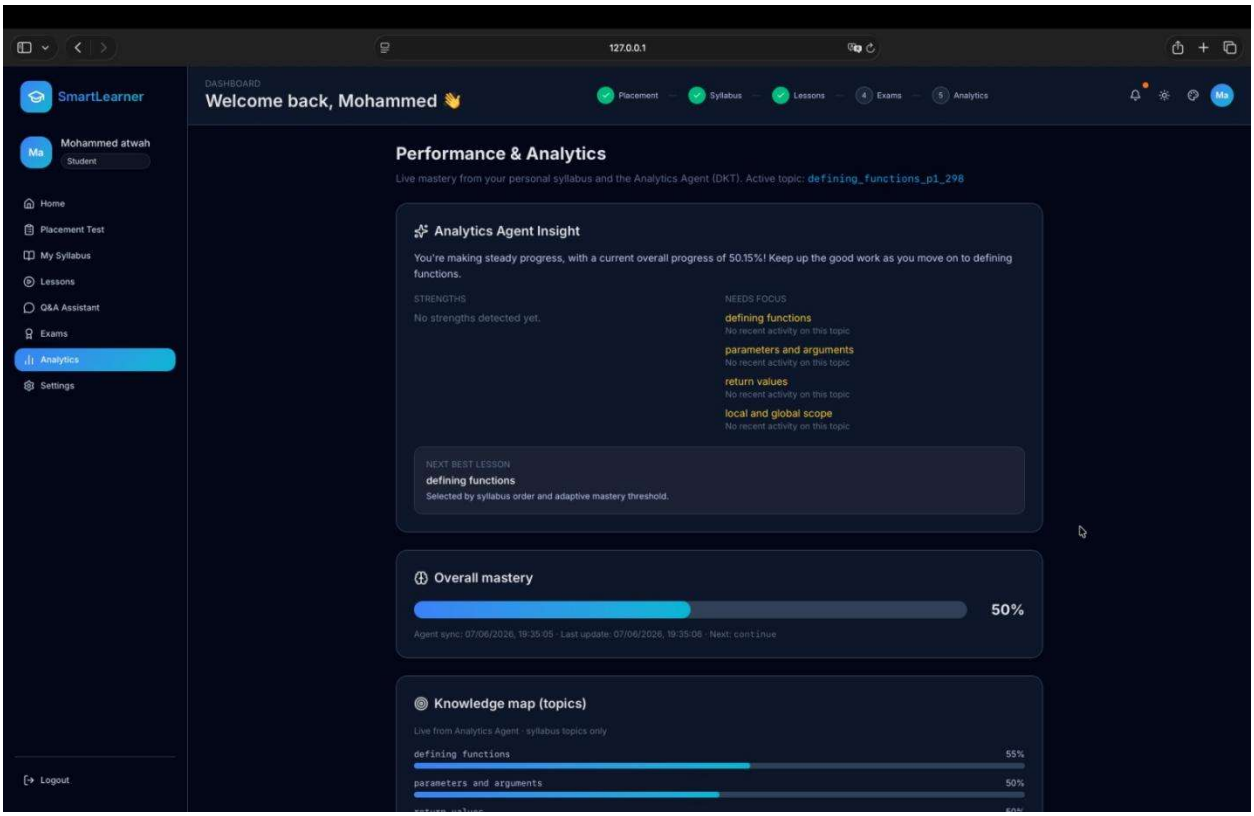
7. Lesson Assessment Quiz(Quiz used to evaluate student understanding after completing a lesson.)



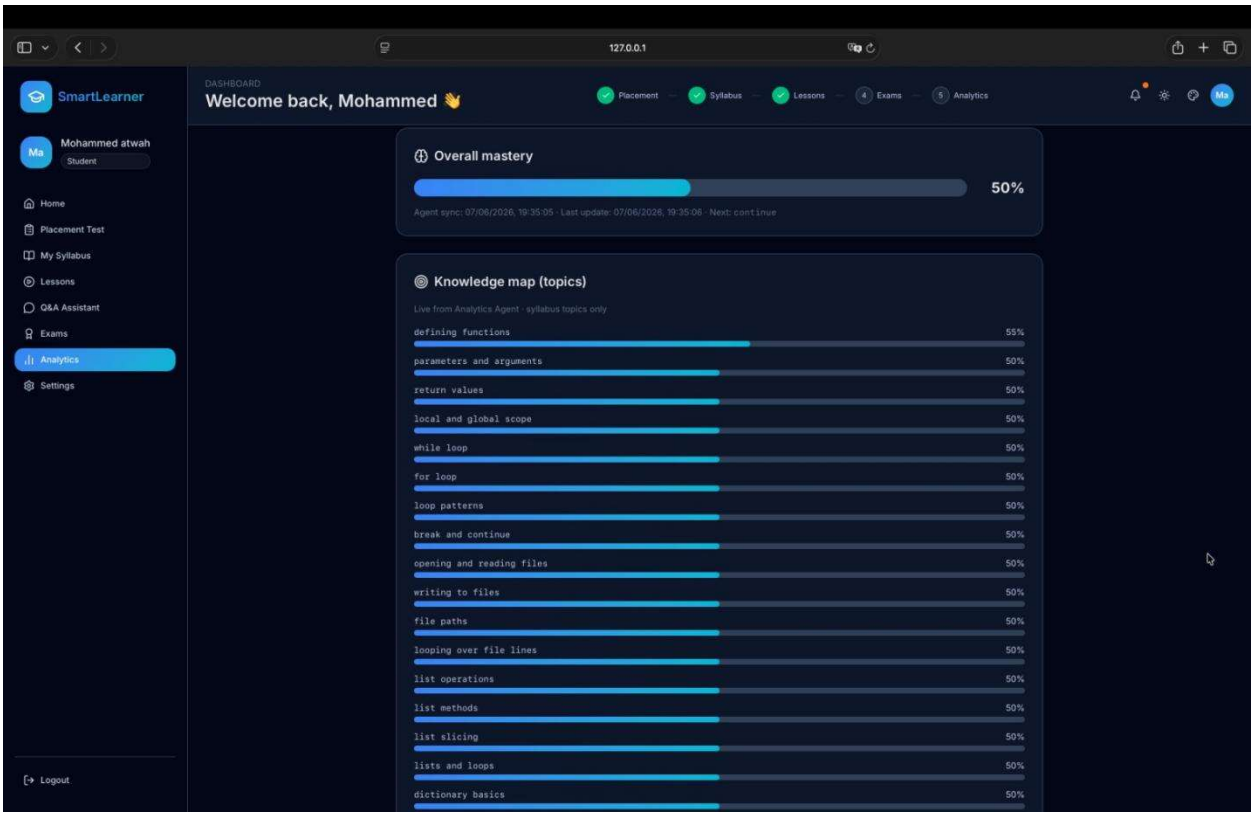
8. Assessment Feedback Popup(Feedback window displaying assessment results and recommended next steps.)



9.Q&A Assistant (AI-powered assistant that answers student questions and provides learning support.)



10. Performance Analytics (Analytics dashboard displaying learning progress and topic recommendations.)



11. Knowledge Map Analytics (Knowledge map showing mastery levels across different course topics.)

Chapter 5

Testing and Evaluation

Testing of an educational platform powered by artificial intelligence and multi-agents provided us with an important insight very early in the process: assessment of an ecosystem of agents is quite different from testing of a web application. Checking if an API call returns HTTP 200 response or if writing into a database works are important, but they don't say anything about the quality of the material created by the agents. In the process of development, we have found out that even such obvious mistakes as generating a placement test question that is clearly aimed at the wrong level of competence could easily slip through. Also, it was not uncommon for our agents to generate lessons written in well-formatted Markdown that nonetheless had a factual mistake or two in them. For this reason, we were forced to create a multilayered approach to assessment that sees content, agents, and reliability as connected parts of a whole. This chapter provides details on how we tested SmartLearner.

5.1 Our Testing Philosophy

Because SmartLearner automatically produces all the elements necessary for students' learning process – placement test questions, tailored syllabus and text of lessons, as well as Q&A and performance reports – content accuracy is now becoming the crucial criterion of our success.

It has become clear that neither automation nor unit testing alone will provide a full spectrum of possibilities to detect all types of bugs. To design a reliable application, we need to use several approaches to assessing it:

• **Automated gatekeeping:** Implement strict runtime validators in order to validate content structure and scope prior to serving the generated text to the user.

• **Deterministic analytics tracking:** Separate out progress metrics and mastery scores from any variation introduced by LLMs.

- **RAG Grounding Guards:** We built relevance gates to verify that context retrieved from core course texts directly matched the user's queries or the lesson's target objectives.
- **Full End-to-End Integration:** We mapped out and tested complete user journeys—from initial registration and placement testing through dynamic lessons, quizzes, and final exams—across both the backend API layers and our frontend interface.
- **Prompt Engineering Experiments:** We built specialized internal scripts to run comparative testing across prompt iterations, logging agent outputs side-by-side against structural rubrics.

- **Peer & Advisor Human Review:** Our team systematically sampled and reviewed generated materials alongside our project supervisor, **Dr. Rasha Obeidat**, evaluating clarity, factual accuracy, and appropriate leveling.

To prove the flexibility of this architecture, we conducted our evaluation using course materials and rubrics from two distinct domains: a **Python Foundations** track (modeled after Python for Everybody) and a **Deep Learning** track. While the subject matter changes, the underlying evaluation logic, validation code, and adaptive loops remain completely track-agnostic.

5.2 Automated Validation: Our Core Quality Gate

The key to ensuring the reliability of SmartLearner is the Generator-Validator design principle behind it. Each major pipeline in the app follows a strict workflow process in four stages: Generate -> Validate -> Log -> Persist.

[Agent Generation] ──> [Deterministic Validator] ──> Pass ──> [Log & Save to DB] ──> Student View

|

Fail

└──> [Fallback/Retry Loop Triggered]

Generated texts that don't pass the validation criteria are flagged, logged, and a fallback system is triggered. Until it passes through the gatekeeper, nothing is shown to the student.

5.2.1 Audit Trail

As a way to trace how agents behaved during the development, we used to log each step of the pipeline into an agent_runs database table, which contained:

□the active agent name and its pipeline stage

□input and output JSON payloads

□validation outcome (pass/fail) and the reasoning

□user ID and timestamps

We found such logging very useful during the development process. For instance, when a syllabus generation request failed because a title generated by an LLM was the exact copy of the main topic name or a placement question generated had duplicate multiple choice options, we wouldn't need to guess.

We would be able to reference the exact log message, connect it to the specific test version of the prompt, and improve the instruction accordingly. We thus made sure that our operational logs became an evaluation tool in themselves.

5.2.2 Deterministic Python Validators

In order not to become a victim ourselves, we deliberately did not apply an LLM to validate an LLM output, but rather came up with fast deterministic validators written in Python.The following table outlines exactly what our validators check across each core component:

Component	Automated Validation Checks & Guardrails
Placement Agent	Confirms exact number of questions to be 5; guarantees four unique options for multiple-choice questions; confirms that correct_answer is contained in choices; verifies that the concept matches the rubric index; detects prohibited keywords based on skill level (like disabling asyncio for the Beginner level); prevents duplicates of question stems.
Syllabus Agent	Ensure that all topics generated adhere to the allowable track schema strictly; lesson_title should not be equal to topic to avoid redundancy; checks that the length of the objective description is more than the minimum required; requires at least three learning objectives and four sub-lessons per chapter.
Exam Agent	Confirms requested question count matches output; verifies 4 unique multiple-choice selections; ensures proper key mapping for answers; guarantees unique question stems across attempts; validates the difficulty enum level.
Lesson Agent	Verifies presence of mandatory source citation phrases; runs text content through an allowlist-based scope validator to ensure the agent didn't drift off-topic.
Q&A Agent	Ensures that the source grounding suffix is present; looks for common code hallucinations based on historical context (for example, Python 2 code structures); includes an obligatory "Quick Check" exercise section if the model did not add it.

Whenever an LLM-based agent struggled or hit an edge case, these deterministic code blocks remained the absolute source of truth, serving as our fallback line of defense.

5.2.3 Runtime Safeguards & Security Gates

In addition to validation of data structures, we integrated scope and safety restrictions into our live runtime workflows:

1. **Input Filtering:** An initial script sanitizes student input prompts against known malicious script injection patterns.
2. **Relevance Filter for RAG:** To save system resources, we implemented an off-topic question gate. In the Q&A loop, a question's relevance is scored based on token overlap with the core set of domain terms; if the student asks something entirely irrelevant, the system automatically refuses it right away.
3. **Code Sandbox:** When executing code samples submitted by students as part of the programming exercises, we use an isolated Python subprocess sandbox, which runs under a restrictive time limit and limited library import list to safeguard our hosting platform.

5.3 Component Evaluation Breakdown

5.3.1 Placement Generator & Validator

For entry-level placement tests, our tests of the component focused mostly on the end-to-end user experience of question evaluation. We tested how the program managed sequential delivery of questions and calculated the assignment of a starting level based on answers.

One early attempt in generating all five placement questions in one go proved inefficient as the generator would often produce duplicates or faulty structures. Ultimately, we settled on a per-slot generation and validated each slot separately, testing whether it could successfully validate and reject overlapping stems and difficulty mismatch between concepts.

To systematically experiment with this process, we created a utility script named run_placement_prompt_experiments.py that allowed us to conduct batch tests on varying prompt versions, save them as-is, run through our PlacementValidatorAgent component, and score the results automatically based on a 1-5 rubric covering five aspects:

Objective Appropriateness: Does it test the required technical concept?

Profundity of Assessment: Does it assess conceptual understanding or only surface-level facts?

Scope Alignment: Does it stay strictly within the bounds of our base textbook track?

Set Coherence: Do the five questions together form a balanced, progressive evaluation?

Tooling Note: This internal rubric evaluator was run on Groq, which is what we use in production for inference, rather than relying on external APIs like GPT-4. It was made for us as a developer tool to evaluate different prompts internally, not as a live runtime monitor.

5.3.2 Syllabus Generator & Validator

For syllabus evaluation, we were particularly interested in identifying and fixing any structural edge cases that gave us trouble during our early builds. Specifically, we fixed these issues:

The "Flat Array" Bug: To ensure that the generator would properly build the correct nested structure of units[] and lessons[], and not break everything into a flat array.

Verbatim Repetition: To detect cases where the lesson name was a verbatim repeat of its parent topic.

The "Practical" Prefix Issue: To get rid of any generic names like "Practical Exercise 1".

Database Mapping: To make sure our valid syllabus JSON can be converted to a proper relation record in PostgreSQL with the right order preserved so we can show the student dashboard.

5.3.3 Lesson Generator & Validator

For generating and validating lessons, we checked how our on-demand text generation works, that Markdown styles work properly, and whether caching is happening as expected – in particular, opening an already cached lesson should immediately pull it from PostgreSQL without making extra, unnecessary calls to the language model.

Additionally, we tested our adaptive remediation logic by simulating a scenario where a student fails a quiz in one of the lessons (getting below the 80% cutoff point). We confirmed that the lesson splitting is handled correctly and that the reassessment is appended at the very end of the last sub-lesson.

5.3.4 Q&A Assistant

This Q&A test also confirmed the ability of the system to dynamically modify the tone and vocabulary according to the user's current mastery level (for instance, employing a simpler explanation for a weaker student). Our mapping and validation included verifying the heuristic routing of queries into one of five different categories: explanation, how-to guides, debugging, architecture comparison, and generic technical queries.

Our quality assessment did not include any abstract theoretical benchmarks such as a generalized "hallucination rate percentage." In contrast, we focused strictly on hands-on comparative reviews.

This included comparing our outputs to the corresponding chunks from the sources used by our RAG system, as well as checking our validation logs to ensure we had not missed any flagged words.

5.3.5 Exam Generator & Validator

We performed tests with all three available versions of the exam generator and validator (three-, five-, and ten-question versions). It has been verified that the system dynamically changes the difficulty of questions depending on the overall student's proficiency and focuses on topics where the student scored worse. Stem deduplication is implemented and enforced.

5.3.6 Analytics Agent & Data Consistency

Where analytics were concerned, the critical issue was always reproducibility. If the inputs and testing history stayed exactly the same, would the numeric outputs always come out the same as well?

[Student Performance Data] ──> [Deterministic Python Logic] ──> Exact Mastery & Progress % (100% Reproducible)

└──> [LLM Text Generator] ──> Personalized Coaching

The reason why our analytics can be completely deterministic is that all the computations behind the students' mastery levels, progress percentages, and velocity scores are done in pure Python code using nothing but mathematical functions. As for the layer that generated the textual output, it only included personalized summary text, coaching prompts, etc., so we made sure that the LLM couldn't influence the core data in any way.

Our first design idea was actually to use the LLM to calculate the mastery scores by prompting it to generate a corresponding number for the provided data. However, when we tried this approach, the outputs turned out to be extremely unstable; even with the exact same performance data, different API queries gave wildly varying results. It was clear to us that relying on the LLM to produce grades and progress percentages was fundamentally flawed..

5.3.7 Retrieval-Augmented Generation (RAG) Service

Instead of judging the performance of the whole RAG system as a conversational agent, we evaluated the performance of our RAG model solely as a retrieval module. The chunks are indexed and fed from the reference files. At run-time, the system does token-overlapping between the query and the content, with weights assigned for particular types of metadata to prioritize matching with chapter titles and sub-lessons.

We manually checked the results of the retrieval against many sets of queries and found that our retrieved top-k results always contained necessary keywords to answer the student's queries. By adjusting the overlapping thresholds, we greatly reduced the number of unnecessary answers flagged by our QA gate and at the same time avoided filtering out relevant student queries.

5.4 Integration and API Testing

For all development phases, the OpenAPI interactive documentation feature (provided by FastAPI at /docs endpoint) was used as a tool for performing integration testing.

Using the OpenAPI Interactive Documentation interface, we performed exploratory testing of our full API route structure where we:

- Checked incoming requests and outgoing responses' exact JSON schema for all main routes on our backend side.
- Tested security-authenticated user paths by providing JSON Web Tokens (JWT) into our authorization header section.
- Caused runtime exceptions in order to verify that our application always provides HTTP-friendly error codes like a 404 Not Found for non-existent entities, proper validation messages for incorrect user input data, and messages about restrictions in case user tries to bypass lesson order restriction.

This combined effort of API interaction checking using OpenAPI and structural python validation ensured that our SmartLearner is reliable, predictable, and correct.

5.4.2 End-to-End User Flows

The following flows were tested repeatedly through the React frontend and directly against the API:

Flow	Main checks
Register / login	JWT issued; dashboard storage scoped per user
Placement	Questions advance correctly; weak/strong concepts stored; final level persisted
Syllabus build	Course created; modules visible; topics match placement level
Lesson open	Generation or cache served; progression lock enforced

Lesson quiz	Pass/fail logic; retry count; sub-lesson split on root lesson failure
Q&A	On-topic vs off-topic routing; formatted answer with Quick Check
Exam	Correct question count; grading logic; optional code run
Analytics	Summary loads correctly; numeric values stable across page refresh

5.4.3 Environment Testing using Docker-based setup

The environment consisting of the backend, frontend, database volume, and pre-packaged RAG assets were verified with the help of Docker Compose on a clean environment. This helped address the recurrent problem of inconsistent environments causing issues when the workflow did not work consistently across different people's environments.

5.5 Human Evaluation

Automated validators identify structural and scope mismatches. They will not spot if an explanation is technically correct but hard to understand, nor if a question is tricky but not violating the specified assessment parameters. Human evaluation was still needed for such judgments.

- **Participants**

Our team worked through the system and iterated their prompts depending on what they observed in generated text content. The project supervisor, Dr. Rasha, provided input on selected outputs based on each project objective including generation of placement questions, syllabi snippets, lessons' paragraphs, and Q&As.

- **Evaluated aspects**

•Is the explanation easy to understand for the designated student level?

•Does the content maintain factual consistency in relation to Python for Everybody or Deep Learning materials?

•Are the assessment questions fair?

•Is the tone suitable for students?

The method consisted of informal peer review during key milestones of development rather than blind review with set rubrics and a statistically sufficient number of participants. Review feedback

contributed directly to making adjustments to the prompts (see Section 4.10), for example the rule of lesson title ≠ topic arose from observing that the titles tended to repeat the topics.

5.6 Prompt Engineering Evaluation

Rather than evaluating the prompt quality once and finalizing it, prompt evaluations were iterative during development cycles and feedback from one round fed the next. The procedure for each agent was essentially:

1.Run the generator with a prompt variant.

2.Analyze the pass/fail rate of the validator and agent_runs logs.

3.Run the placement experiment script with manifest variants optionally scored by rubric.

4.Manually read through the outputs.

5.Decide whether to accept or reject the prompt with explanation per Section 4.10.

Prompts were rejected based on quantitative criteria:

Batch placement prompts would yield duplicate stems;

Flat syllabus prompts would violate the hierarchical structure of the validator;

Lesson prompts with too few words would create trivial content;

Text-only question-answer prompts would not parse or validate;

Analytics numbers calculated by the LLM would vary between runs.

The rejection criteria were not subjective it appeared as outright failures in the validator or in agent_runs logs.

5.7 What was in scope

It is important to clarify what our evaluation did and did not entail.

Evaluation scope:

•Functional correctness of the orchestration and API layer.

•Validator and rubric enforcement across all agents.

•Content grounding via RAG and guardrails.

•Adaptive behavior: placement tiers, weak topic syllabus personalization, remediation split, difficulty distribution for exams.

•Deterministic analytics generation.

•Qualitative assessment of generated content by the team and supervisor.

Hence our evaluation substantiates architectural correctness, validation and prototype behavior. It does not substantiate learning outcome improvements compared to a control group since that would need a proper study that went beyond our graduation project.

5.8 Summary

SmartLearner was tested using a suite of automated Generator-Validator pipelines, deterministic rubric and runtime guards, logging of agent runs, API and end-to-end manual testing, prompt experiments offline, and team and supervisor qualitative review.

The strategy was based on the real-world risks faced by this application: the primary failure mode is not loss of server connectivity but rather low quality or off-target generation. Rubrics and validators offered a reliable way to gate the process. Humans were needed for the educational judgment that was impossible to automate.

Chapter 6

Conclusion

Chapter 6 summarizes the SmartLearner project by integrating all the design, development, and evaluation presented in previous chapters. From the very beginning, the idea was not to create another system that presents a standard, predetermined program to students without any modifications; instead, we set the target of designing an approach that can adapt to each student's needs. In this chapter, we look back at whether we managed to achieve our initial goal, discuss the evaluation findings, and consider the possible directions for further work.

Unlike those systems that provide the same course to each student, SmartLearner offers an individual profile based on adaptive placement, a personalized curriculum built based on the results of the placement, as well as adaptive content of lessons, tests, and analytics tailored to the student's level and weaknesses. Two tracks are available in SmartLearner, Python for Everybody and Deep Learning. Yet, the process of adaptation in SmartLearner is independent of these learning tracks – it uses information about the student acquired from his/her activity within the platform. Chapter 1 through Chapter 3 introduced the design ideas. Chapter 4 offered information about the system's implementation. Evaluation of SmartLearner is discussed in Chapter 5.

6.1 Contributions and Achievements

The key contribution of this project lies in a fully developed multi-agent adaptive learning platform rather than in any design concepts or proof-of-concepts. It was implemented, deployed, and thoroughly tested, with end-to-end execution demonstrated. This project delivered:

•Orchestrator Service managing all the agents via a hub-and-spoke model, where all workflows go through Generate → Validate → Log → Persist and leave a trace in the agent_runs table;

•Adaptive learning process, which goes from placement through personalized syllabus creation, lesson delivery, quizzes and assessments, remediation and analytics, based on insights gained from each step;

•Agent-based generators and validators for placement, syllabus, lessons, Q&A, and exams, which do just one thing apiece and can be individually updated or replaced without disrupting anything else;

•RAG pipeline supporting all the generations done using course materials prepared ahead of time, not arbitrary outputs produced using an LLM.

•The hybrid approach, in which all numeric parameters, including levels of mastery, percent completed, rate of learning, and risk factors, are computed by means of a deterministic algorithm implemented in Python, whereas the narrative text is generated by the LLM;

•Run-time guardrails like prompt sanitization, RAG filters, scope constraints, and contingency plans in case of API outages or other problems;

•Cycle of prompt engineering resulting in validated and rubric-compliant prompts for all agents, with adoption tracking and rationale provided.What sets SmartLearner apart from a simple learning management system is that the course isn't a set course of action. Syllabi, the amount and depth of coverage, difficulty of quizzes and assessments, and answers provided during Q&A all depend on the results of the placement and current student's performance. Different students even taking the same path would have different experiences.

6.2 Evaluation Summary

Not crashing the server is not our main failure mode. Instead, we may fail by delivering content to the student in a format inappropriate for what they need: a placement question testing a concept other than the one it should, a lesson that is perfectly formatted but inaccurate, or an explanation that is too difficult or too simple. We figured out that the usual software metrics would not detect that kind of problem.

Thus, we combined several types of testing for coverage of different aspects of the system. We wrote automated validators that made sure that content met certain structural and scope rules before being delivered to a student. To keep track of everything that was happening, we used the agent_runs logging file which kept record of everything we had generated and what had been refused by our system. Then, we tested our end-to-end flows, exercising our application via the live API with our React frontend, trying out every type of user journey from registration to final analytics generation. For our prompt engineering, we compared variants of prompts we used for placements offline using the outputs we had saved and an LLM rubric scoring tool. Finally, our team and our supervisor also did a manual review of some of the generated content for clarity and level appropriateness.

Adaptive nature of the system was the thing we were most interested in verifying. We double-checked that both weak and strong concepts selected by the placement test feed into the syllabus personalization properly. Also, we checked whether failing a root lesson quiz triggers a dynamic split into the appropriate sub-lessons for retesting instead of just repeating it again. Finally, we checked that the exam generation code does the right priority on weak topics and de-duplicates question stems when generating a new test. We verified that by calculating things in a pure Python code to be fully reproducible and deterministic.

Our evaluation's boundaries are that it allows us to make claims only about how our system works but not the improvements it makes. We recognize that gathering that kind of evidence would

require a controlled study with a larger student sample, which remains our most important next step for this project.